

mAX Core

김 화 종

hjkim3@gmail.com

김화종

- [학력]**
- 서울대학교 전자공학 학사
 - KAIST 석사/박사 (디지털 신호처리)

- [경력]**
- **현재:** 한국제약바이오협회 K-MELLODDY 사업단장
 - **전:** 강원대학교 컴퓨터공학과 교수,
KAIST IT융합연구소 겸직교수

- [강의]**
- **AI 부트 캠프:** 한전, 한전 KDN, LG전자, KB국민금융, NH농협금융, LS그룹, LG화학, 하림, 두산인프라코어, 두산산차, 고려해운, 동아ST

- [저서]**
- 파이썬으로 배우는 머신러닝, 홍릉과학출판사, 2022
 - 한국생산성본부 데이터사이언티스트 자격증 DSAC 교재 1~4권, 2019
 - 데이터사이언스개론, 홍릉과학출판사, 2014 (학술원 우수도서 선정)
 - 컴퓨터 네트워크 프로그래밍, 홍릉과학출판사, 2012 (80여 대학 교과서)



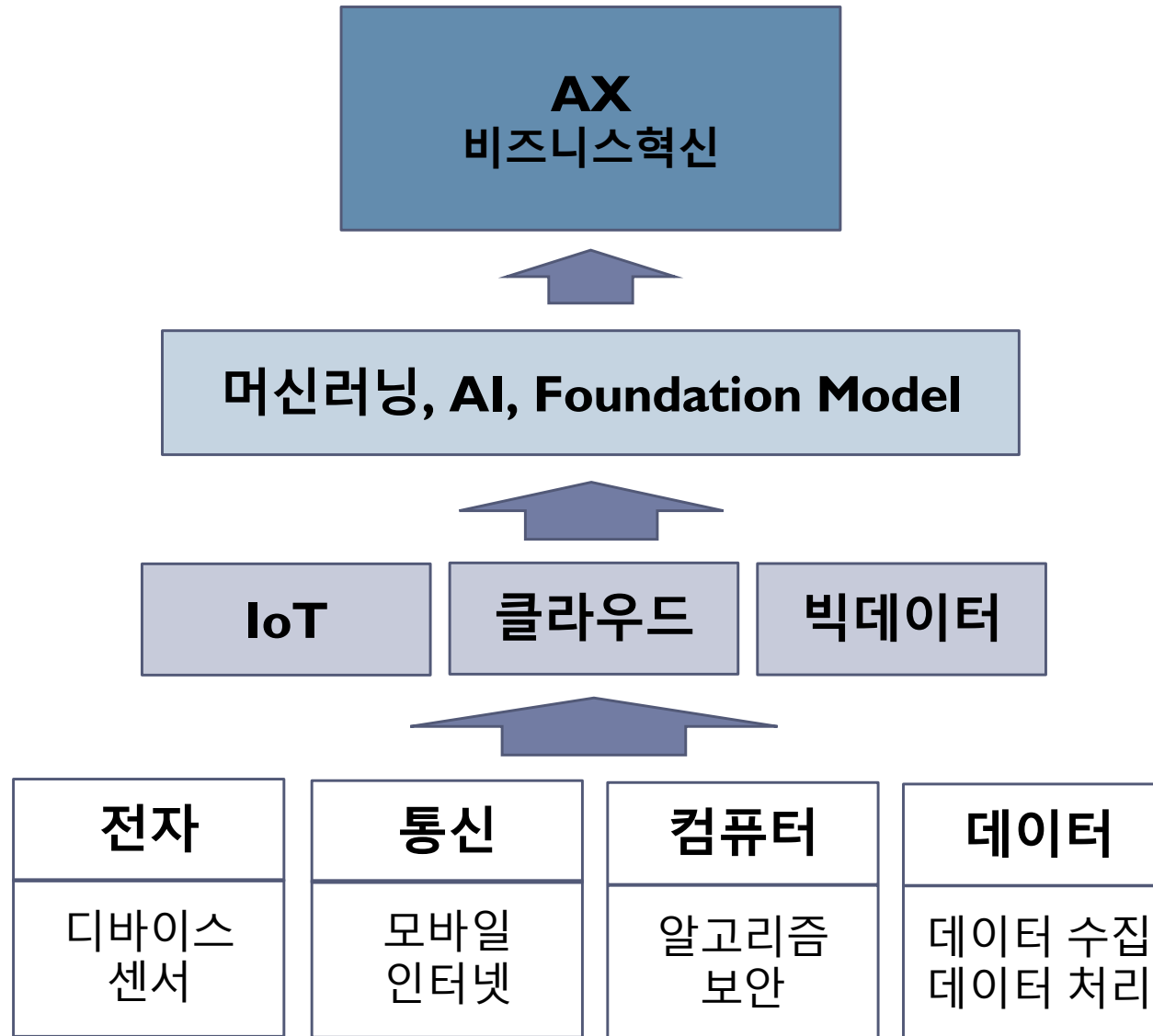
AI 개요

AI 범위

머신러닝	딥러닝	AI	LLM	파운데이션 모델	FM 활용	AX
확률 기반 지도학습 비지도학습	신경망 (CNN/RNN) 표현 학습	Transformer Attention 메커니즘	GPT · BERT 수십억 파라미터	멀티모달 제로샷 / 퓨샷	RAG Fine-tuning Prompt Eng.	AI 에이전트 조직 · 업무 재설계
스팸 필터 추천 시스템	이미지 인식 음성 인식	기계 번역 감성 분석	챗봇 · QA 문서 요약	이미지 생성 멀티모달 검색	기업 지식검색 AI 코파일럿	산업 AI 전환 비즈니스 혁신

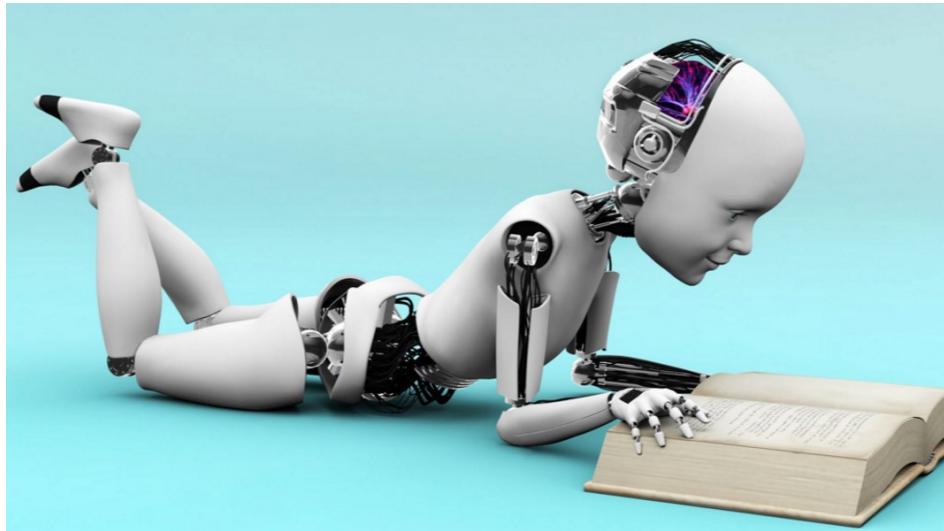
※ AX(AI Transformation): AI 기술을 통한 업무 · 비즈니스 전반의 구조적 전환

AX (AI Transformation)



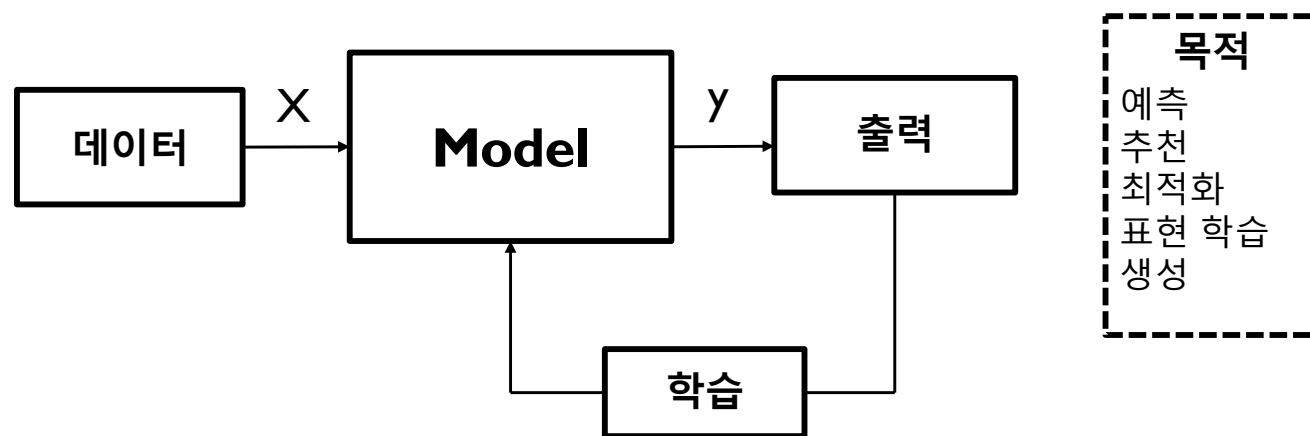
머신러닝 (machine learning)

- ▶ AI 구현의 기반 기술
 - ▶ 컴퓨터가 (인간처럼) 데이터를 보고 스스로 학습한다
- ▶ 사람의 지식이 개입되지 않는, end-to-end 방식으로 동작
 - ▶ 논리적인 해석이나 중간 값 확인을 필요로 하지 않는다
 - ▶ 사람은 **학습하는 기계를** 발명한 것



AI Model

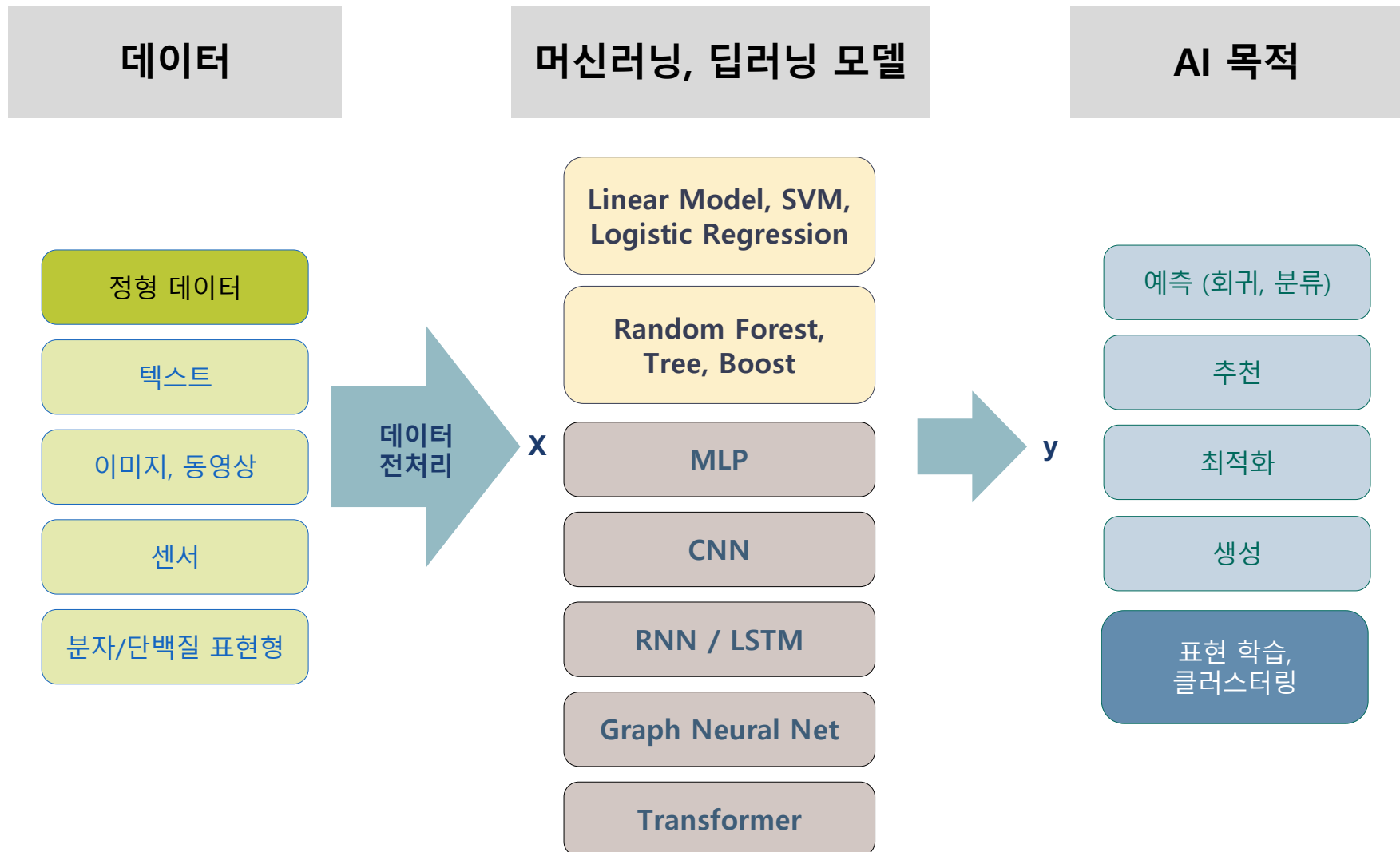
- ▶ AI 구현의 핵심은, 데이터(X)로부터 원하는 출력(y)를 생성하는 모델(Model) SW 개발이다
- ▶ 수식은 가장 명쾌한 모델이나 생명현상을 포함한 복잡한 실제 세계를 모델링하는데 한계가 있다
- ▶ 데이터를 기반으로 동작하는 **Machine Learning Model**이 현재 AI 구현에 널리 사용되고 있다
 - ▶ 데이터: 학습 데이터, 검증 데이터, 테스트 데이터



AI Model 유형

예측 모델 Predictive Model	입력 데이터(X, feature)를 보고 출력(y, 레이블) 예측 → 이상치(OOS) 예측, 배치 품질 예측	$X \rightarrow \text{Model} \rightarrow y$
추천 모델 Recommender Model	최적의 상품·처방·후보 추천 → 신약 후보 추천, CAPA(시정 및 예방 조치) 대응책 추천	$X \rightarrow \text{Model} \rightarrow \text{candidate}$
최적화 모델 Prescriptive Model	현재 조건(X)에서 최적 설계·제어 변수 도출 → 공정 핵심변수(CPP) 최적화, 배합 최적 설계	$X \rightarrow \text{Model} \rightarrow \text{control}$
표현 학습 Representation Learning	X를 학습하여 의미 있는 벡터 표현(embedding) 학습 → 파운데이션 모델(LLM), 분자 임베딩	$X \rightarrow \text{Model} \rightarrow \text{vector}$
생성 모델 Generative Model	파운데이션 모델을 이용하여 원하는 데이터(X [^]) 생성 → 자연어 처리, 분자 생성 모델, 단백질 구조 생성	$X \rightarrow \text{Model} \rightarrow X^{\wedge}$

AI 구현 체계



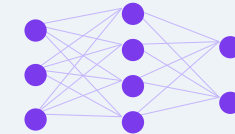
머신러닝 모델

머신러닝 유형		알고리즘	특징
지도학습 (y가 존재)	선형 계열	선형 모델, SVM 로지스틱회귀	곱셈과 덧셈으로 점수를 구하고 이를 이용하여 회귀와 분류 예측
	신경망	MLP, CNN, RNN, Transformer Graph Network	매트릭스 연산을 기반으로 점수를 계산하며 활성화 함수 도입
	트리 계열	결정 트리, 랜덤포레스트, 부스팅	True/False 선택을 반복하여 회귀와 분류 예측 수행. 스케일링이 필요없다
	기타	kNN, 베이지	특성 공간상의 거리를 기준, 또는 조건부 확률을 기준으로 예측
비지도학습 (y가 없음)	클러스터링	k-means, DBSCAN	특성 공간상 거리 또는 유사도를 기준으로 샘플을 그룹핑
	데이터 변환	스케일링, 로그변환, 카테고리 인코딩	모델의 성능을 높이 위한 데이터 전처리
	차원 축소	PCA, t-SNE	계산량을 줄이고 모델 성능을 향상, 또는 시각화를 위한 차원 축소
	표현 학습	LLM 등 파운데이션 모델	입력 데이터를 latent vector로 표현하는 비지도 학습
강화학습		시행착오의 경험을 누적	샘플 입력마다 정답이 있지 않지만 최종적으로 달성해야 할 목표를 알려주고 보상을 통해 학습

딥러닝 모델

MLP

Multi Layer Perceptron. 완전 연결(Fully Connected) 레이어들의 다층 구조로 **비선형적인** 입출력 관계 예측에 주로 사용



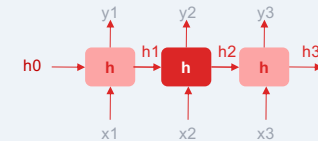
CNN

Convolution Neural Network. 2차원 이미지, 1차원 시계열 데이터의 **패턴 추출**에 주로 사용



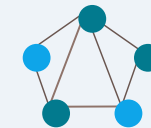
RNN (LSTM)

Recurrent Neural Network (Long Short Term Memory). 순환 구조로 **시퀀스 데이터**· 텍스트 분석 · SMILES 분석 등에 활용



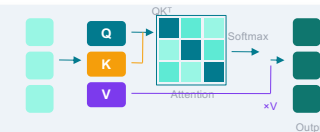
Graph Model

노드와 엣지로 구성된 그래프 형태의 모델로 **분자 구조**, 사회관계망 등의 분석에 사용



Transformer

입력 시퀀스를 블록 단위로 나누어 Self-Attention을 계산하는 방식으로 텍스트, 시퀀스 데이터의 **문맥 파악** 성능을 향상. LLM의 성능 고도화 (BERT, GPT)



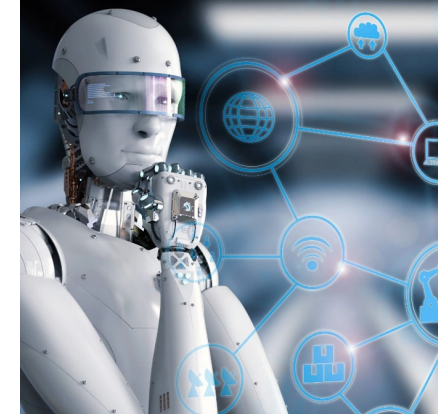
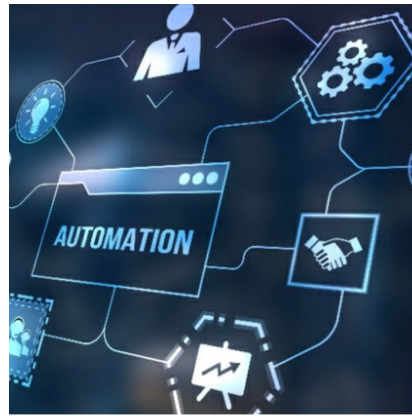
AI 도입 전략

DX/AI 이용자 니즈

- ▶ 외부 고객의 니즈 (서비스/제품에 대한)
 - ▶ 비용과 시간 절감
 - ▶ 안전성 (UI 개선)
 - ▶ 오동작(실수)을 커버하는 인터페이스
 - ▶ 편리성 (UX 개선)
 - ▶ 설명이 필요 없는 인터페이스
 - ▶ 즐거움 (CX 개선)
 - ▶ Wants → Likes → Stickness
- ▶ 내부 고객의 니즈 (ML 솔루션에 대한)
 - ▶ 활용시의 **실제적인 효과** (정확도 개선, 비용과 시간 절감)
 - ▶ 도입 및 운영의 **편리성** (데이터 입력의 편리성 등)
 - ▶ **투자비용** (개발비, 장비, 교육비 등)

AI Roadmap

Digitization → Governance → Automation → AI



- 문서의 디지털화로 오류 최소화
- 데이터, 프로세스 표준화
- 관리의 자동화
- 실시간 분석, 대응
- 품질 관리
- 예지 보수

DX/AI 적용 범위

1. 기획 (서비스/제품)
2. 수요 조사 (이용자 분석)
3. 설계 (매뉴얼 관리)
4. 시험 (시간 단축)
5. 최적 제어 (비용과 효율 개선)
6. 장애 예측 (장비, 부품, 제품 품질 관리(QC))
7. 수요 예측, 생산 계획, 재고 예측, 물류 최적화 (SCM)
8. 마케팅, 서비스/제품 추천 (CX)
9. 피드백 분석 (QA)

의약품 제조 AI Stack

거버넌스 / 컴플라이언스	Audit Trail · Data Governance EDMS / QMS · Secure Computing
응용	Smart Manufacturing · Digital Twin · Predictive Maintenance PAT Optimization · Regulatory Intelligence · QA Intelligence Model Lifecycle · R&D Innovation
모델 / AI 엔진	Machine Learning · Generative AI · Physics-informed ML RAG · Agentic AI · Foundation Models · Bio-Vertical LLM
데이터 플랫폼	GxP-compliant Data Platform · IoT · LIMS MES · Historian · eBR / eBMR
컴퓨팅 인프라	Cloud · Edge Computing

AX 추진 특징

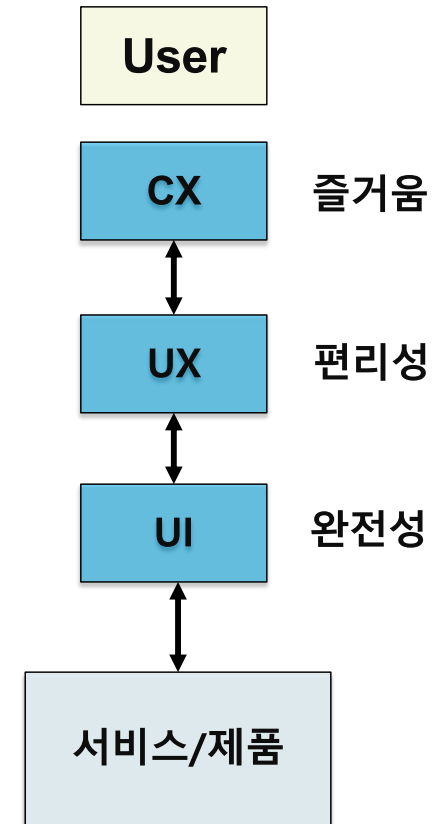
- ▶ AX 도입 진입장벽이 낮아지고 있다
 - ▶ 네트워크 비용, 클라우드를 이용한 고속 연산 비용 감소
 - ▶ 공개 소프트웨어 사용으로 코딩의 진입 장벽이 낮아졌다
 - ▶ 기업간 경쟁이 치열하다
 - ▶ 그러나 어디서, 어떻게 시작해야 할지가 어렵다
- ▶ AX 추진 방법은 기업마다 다르다
 - ▶ 기업마다 비즈니스 모델, 보유한 데이터가 다르다
 - ▶ 의사결정자, 관리자, 실무자의 이해도가 서로 다르다

우수 AI 서비스 요건

- ▶ 제대로 동작하는 것
 - ▶ AI의 동작은 100% 완벽하지 않다, acceptable range 필요
- ▶ 사람의 needs, wants, likes를 만족하는 것
 - ▶ 고객이탈, 피드백 분석
- ▶ 편리한 UX과 사용성
 - ▶ 세가지 단계
 - ▶ Explanation Required - Self Explanatory - Self Evident
- ▶ Stickness, delight
 - ▶ 오늘의 놀라움은 내일의 평범함 - AI effect

UI/UX/CX

- ▶ UI
 - ▶ 오류없는 동작 추구
 - ▶ 인터페이스 시각 디자인 중심
- ▶ UX
 - ▶ 사용자의 편리성 추구
 - ▶ 직관적 인터페이스 중심 (매뉴얼이 필요 없게)
- ▶ CX
 - ▶ 즐거움 추구, stickness
 - ▶ 고객 의도 맞춤형 서비스 중심
- ▶ AI 접목으로 **지능화된** UI/UX/CX를 기대
 - ▶ 사용자의 기대 수준이 점차 높아진다



AX 프로세스

사전 조사

- ▶ 본인의 소속과 업무
- ▶ 가장 성공적으로 보이는 국내외 AX 사례 (1~2개)
- ▶ 현업의 당면한 가장 큰 애로사항은? (1~2개)
 - ▶ AI, 디지털 기술과 관계 없이
 - ▶ 해결방안이나 사용할 기술에 관계없이 도출
- ▶ 앞으로 어떤 업무에 AI를 적용하고 싶은지?

AX 4단계 (PDMA)

Problem

1 문제 정의

- 해결해야 할 문제가 명확히 무엇인가
- 데이터 분석과 AI로 해결할 수 있는 문제인가
- 어떤 데이터를 사용하여 (X), 어떤 값을 예측, 추천, 제어할 것인가(y)
- 최종적으로 얻는 이득은 무엇인가

Data

2 데이터 파악

- 관련 데이터에 어떤 것들이 있는가
- 타부서의 다른 업무 데이터는 관련이 없는가
- 학습 데이터는 충분히 확보 가능하며 누구의 도움이 필요한가
- 데이터의 지속적 확보 방법과 비용은 적절한가

Model

3 전처리·모델개발

- 여러 모델 비교 후 어떤 모델을 사용할 것인가
- 성능평가 지표는 무엇이며 새로운 지표 정의가 필요한가
- 데이터 전처리 방법과 데이터 추가 확보 방안은
- 누가 개발할 것이며 어떤 검증 체계가 필요한가

Application

4 배포·확산

- 현장 적용 시나리오가 타당한가 (누가 어떻게 사용하나)
- 실시간 데이터 공급 및 지속 운영이 가능한가
- 피드백 수렴과 기능 개선 계획은
- 응용 확산 및 상용화가 가능한가

문제정의서

프로젝트명	
작성자 / 작성일	
현재 문제 상황 (As-Is)	현재는 어떻게 처리하고 있나
원하는 목표 상태 (To-Be)	
프로젝트 정의문	어떤 문제를, 어떤 데이터를 사용해서, 어떻게 해결할지 평이한 문장으로 기술
성공 지표 (KPI)	최종적으로 얻는 이득은 무엇인가
관련 데이터 목록과 특성	
예상 리스크	프로젝트가 실패할 수 있는 리스크, 3개 이상
협력 부서, 담당자	

데이터분석서

프로젝트명 / 작성자	
핵심 데이터 목록	
각 데이터별 특성	수집기간, 데이터 크기, 컬럼,
타겟 변수 (Label) 정의	예측하고자 하는 타겟 변수(y)가 있는가? 없으면 새로 정의 가능한가?
관련 데이터 목록 및 특성	어느 부서에 있는가, 확보 가능한가
데이터 확보 가능성	

모델개발서

프로젝트명 / 작성자	
입력과 출력의 정의	x와 y가 명확히 정의되었는가
데이터 전처리	어떤 전처리가 필요한가
모델 구축	어떤 모델을 사용할 것인가 (회귀와 분류)
모델 검증	훈련과 검증 데이터 split
문제 재정의	해결할 문제, x와 y를 재정의 할 것인가
추가 데이터 확보	어떤 데이터가 더 필요한가
모델 성능 평가와 개선	
누가 개발할 것인가	

활용계획서

프로젝트명 / 작성자	
사용 시나리오	누가, 어디서, 어떻게 사용하는가
배포, 운영 계획	누가 설치, 운영 할 것인가
장비, 예산	필요한 장비와 예산은
성과 및 피드백 분석 계획	원하는 성과를 얻고 있는가
데이터의 지속적 확보 계획	
개선 및 확산 계획	타 분야의 확산 또는 상용화 계획

체크 리스트

과제 도출 Check Lists

- ▶ 단순반복 작업으로 시간을 소모하거나 오류가 많은 작업은?
- ▶ 더 빠른 처리가 필요한 작업은?
- ▶ 기존의 방법에 분명한 한계가 있어 개선이 필요한 문제는?
- ▶ 기존에는 이 문제를 어떻게 해결하고 있었나?
- ▶ 경쟁사, 타업종에서 추진한 (유사한) 사례가 있는가?
- ▶ 문제의 크기가 적당한가? 너무 크거나 작은 문제는 아닌가?
- ▶ 과제를 세 문장으로 명확히 표현할 수 있는가?
 - ▶ 어떤 문제를, 어떤 방법으로 해결하여, 어떤 서비스를 제공한다
- ▶ 솔루션의 핵심 기능은 무엇인가?

역량 분석 Check Lists

- ▶ 우리 회사가 경쟁력이 높은 이유는 무엇인가?
- ▶ 우리 회사에서 확보하고 있는 가치있는 데이터는 무엇인가?
- ▶ 팀의 역량이 축적될 수 있는 과제인가?
- ▶ 내부 개발자가 있는가?
- ▶ 역량강화 프로그램이 있는가?
- ▶ 외부 협력 전략이 있는가?

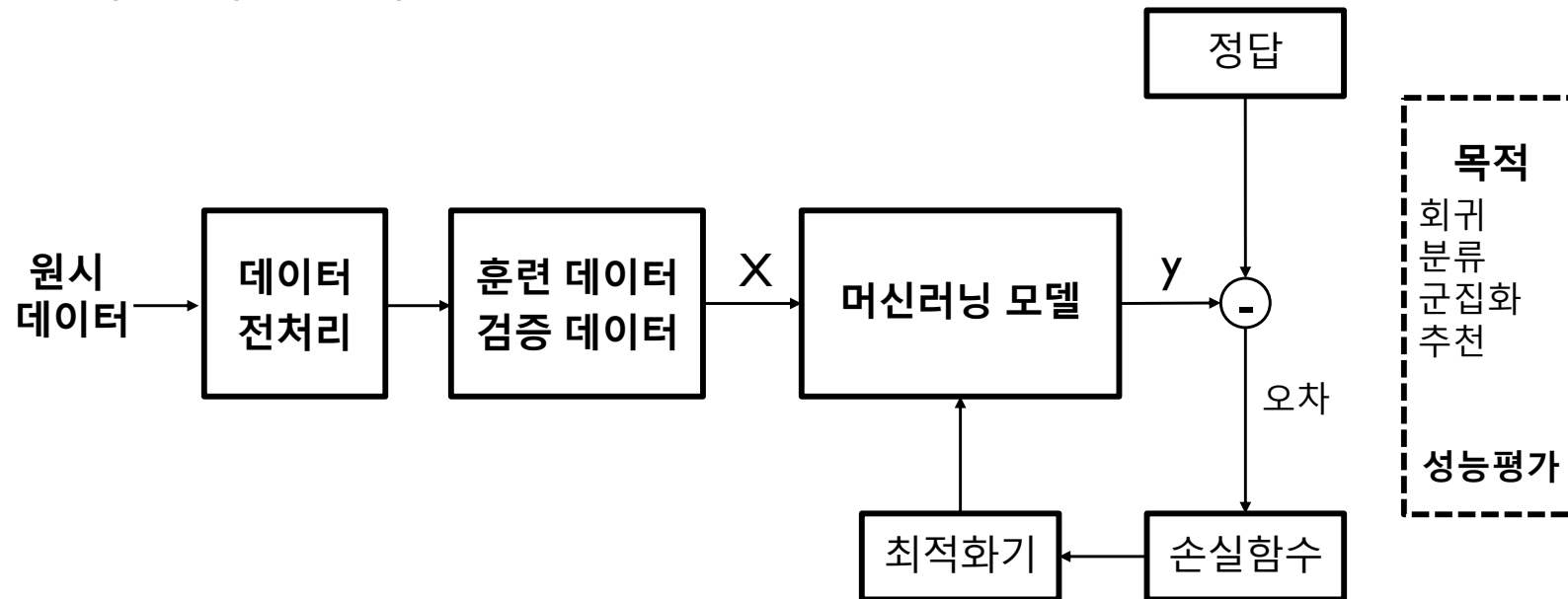
추진 Check Lists

- ▶ 개발팀을 어떻게 구성할 것인가?
- ▶ 핵심 관련 부서와의 협조 방법은(데이터, 인력, 장비, 운영 등)?
- ▶ 규모가 작지만 사내 파급효과가 있는 파일럿 프로젝트인가?
- ▶ 이해 관계자가 초기에 모두 참여했나?
- ▶ 사용자 이용 시나리오를 자세히 점검해보았나?
- ▶ 이용 데이터 수집 방법을 수립했나?
- ▶ 예상 리스크를 세 개 이상 찾았는가?

AI 구현 기술

머신러닝 모델 구현

- ▶ 데이터 전처리
 - ▶ 모델의 성능을 높이기 위해서 결측치 처리, 이상치 처리, 스케일링, 카테고리 인코딩 등을 수행하는 것
- ▶ 훈련/검증 데이터
 - ▶ 모델을 만드는 데는 훈련 데이터를, 성능을 검증하는 데는 검증 데이터를 사용한다



머신러닝 모델

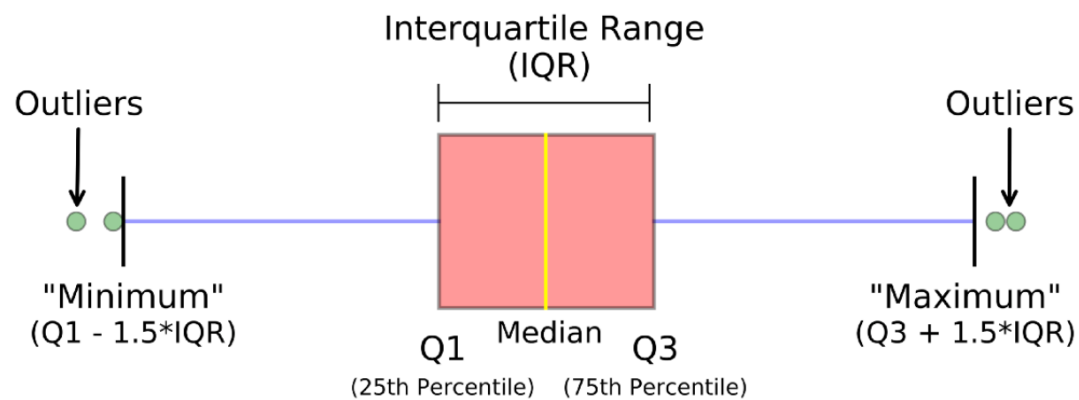
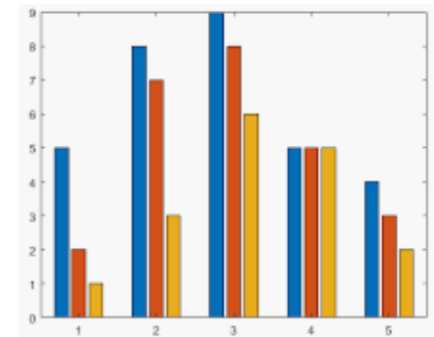
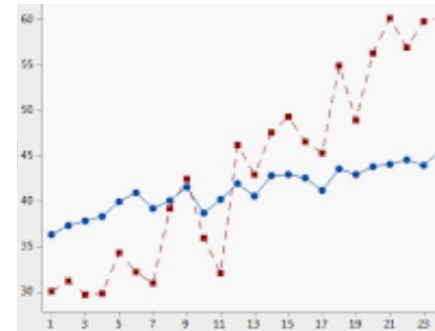
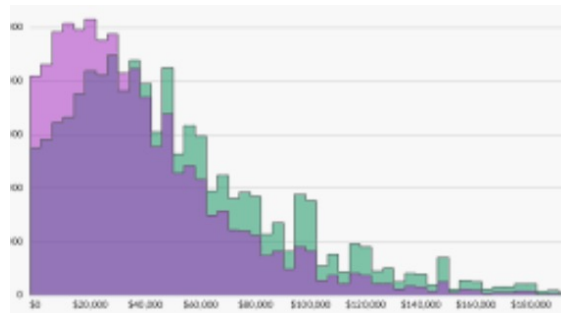
머신러닝 유형		알고리즘	특징
지도학습 (y가 존재)	선형 계열	선형 모델, SVM 로지스틱회귀	곱셈과 덧셈으로 점수를 구하고 이를 이용하여 회귀와 분류 예측
	신경망	MLP, CNN, RNN, Transformer Graph Network	매트릭스 연산을 기반으로 점수를 계산하며 활성화 함수 도입
	트리 계열	결정 트리, 랜덤포레스트, 부스팅	True/False 선택을 반복하여 회귀와 분류 예측 수행. 스케일링이 필요없다
	기타	kNN, 베이지	특성 공간상의 거리를 기준, 또는 조건부 확률을 기준으로 예측
비지도학습 (y가 없음)	클러스터링	k-means, DBSCAN	특성 공간상 거리 또는 유사도를 기준으로 샘플을 그룹핑
	데이터 변환	스케일링, 로그변환, 카테고리 인코딩	모델의 성능을 높이 위한 데이터 전처리
	차원 축소	PCA, t-SNE	계산량을 줄이고 모델 성능을 향상, 또는 시각화를 위한 차원 축소
	표현 학습	LLM 등 파운데이션 모델	입력 데이터를 latent vector로 표현하는 비지도 학습
강화학습		시행착오의 경험을 누적	샘플 입력마다 정답이 있지 않지만 최종적으로 달성해야 할 목표를 알려주고 보상을 통해 학습

데이터 탐색

- ▶ **탐색적 분석:** exploratory data analysis (EDA)
- ▶ 본격적인 데이터 분석이나 머신러닝을 수행하기에 앞서 데이터의 전체적인 특성을 살펴보는 것
 - ▶ 분포, 패턴, 동향 파악
- ▶ 수집한 데이터가 분석에 적절한지 알아보는 과정
 - ▶ 오류가 얼마나 많은지
 - ▶ 데이터가 통계적 가정(계층적 샘플링, stratified sampling)을 만족하고 있는지
- ▶ 시각화 기술을 사용

시각화 함수

- ▶ 히스토그램
- ▶ 타임 플롯
- ▶ 바플롯
- ▶ 박스플롯
- ▶ 산포도

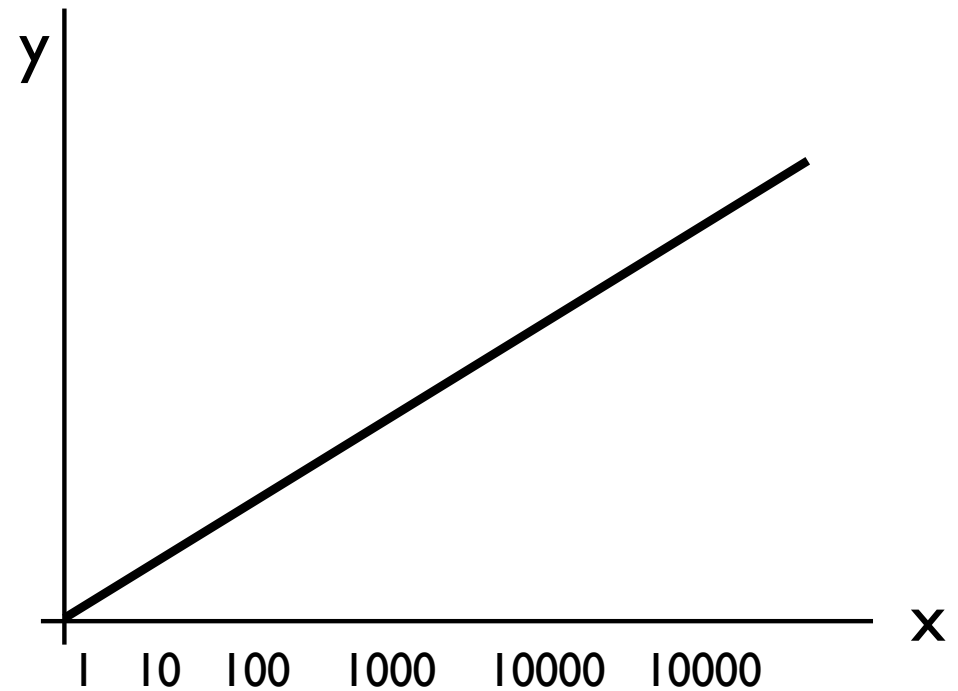
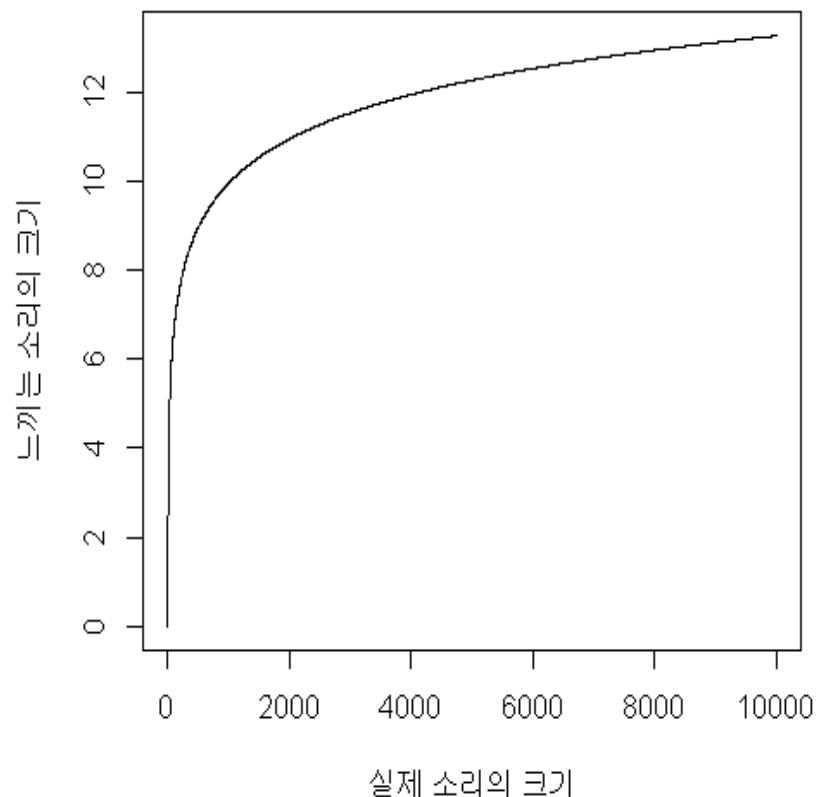


데이터 전처리

- ▶ 데이터 정제 (cleaning)
 - ▶ 빠진 값을 처리하거나 오류를 수정하는 것
- ▶ 이상치 처리
 - ▶ 비정상적으로 튀는 값(이상치)을 찾거나 제거하는 것
- ▶ 데이터 변환 (transformation)
 - ▶ 로그, 역수, 승수, 카테고리 변수를 적용하는 것
- ▶ 스케일링 (scaling)
 - ▶ 여러 데이터의 특성 값의 범위를 동일한 조건으로 조정하는 것
- ▶ 특성 선택과 차원 축소
 - ▶ 특성 중 일부를 선택하거나 특성을 조합하여 새로운 특성을 만드는 것
 - ▶ 주성분 분석(PCA): 머신러닝에 사용할 특성의 수를 줄이는 것
 - ▶ Principal components analysis

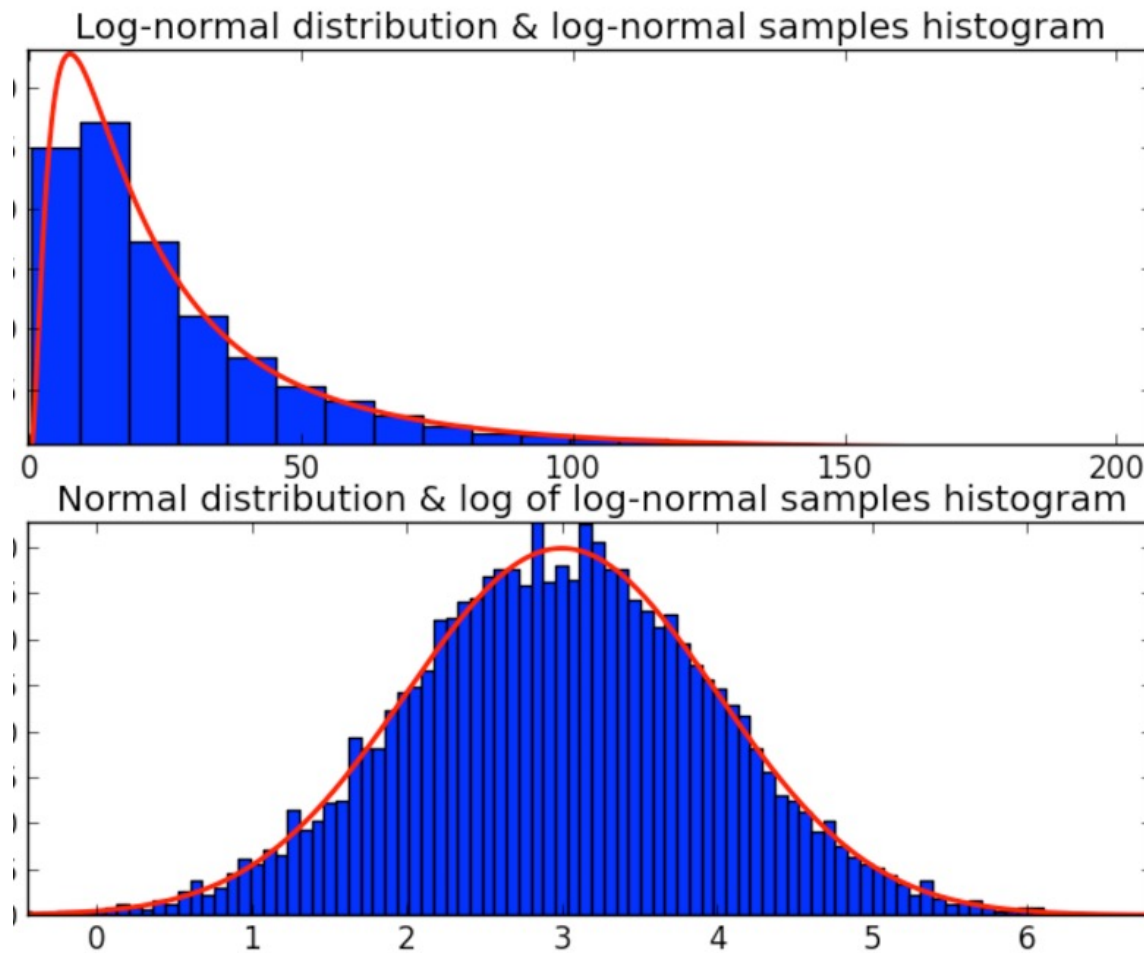
로그 변환

- ▶ 자극에 대한 반응이 현재 보유한 양 x 에 반비례하면 기울기가 $1/x$ 이 되고 이는 로그 함수에 해당한다
- ▶ 로그를 취한 값에 대해서는 자극-반응 관계가 선형적이 된다

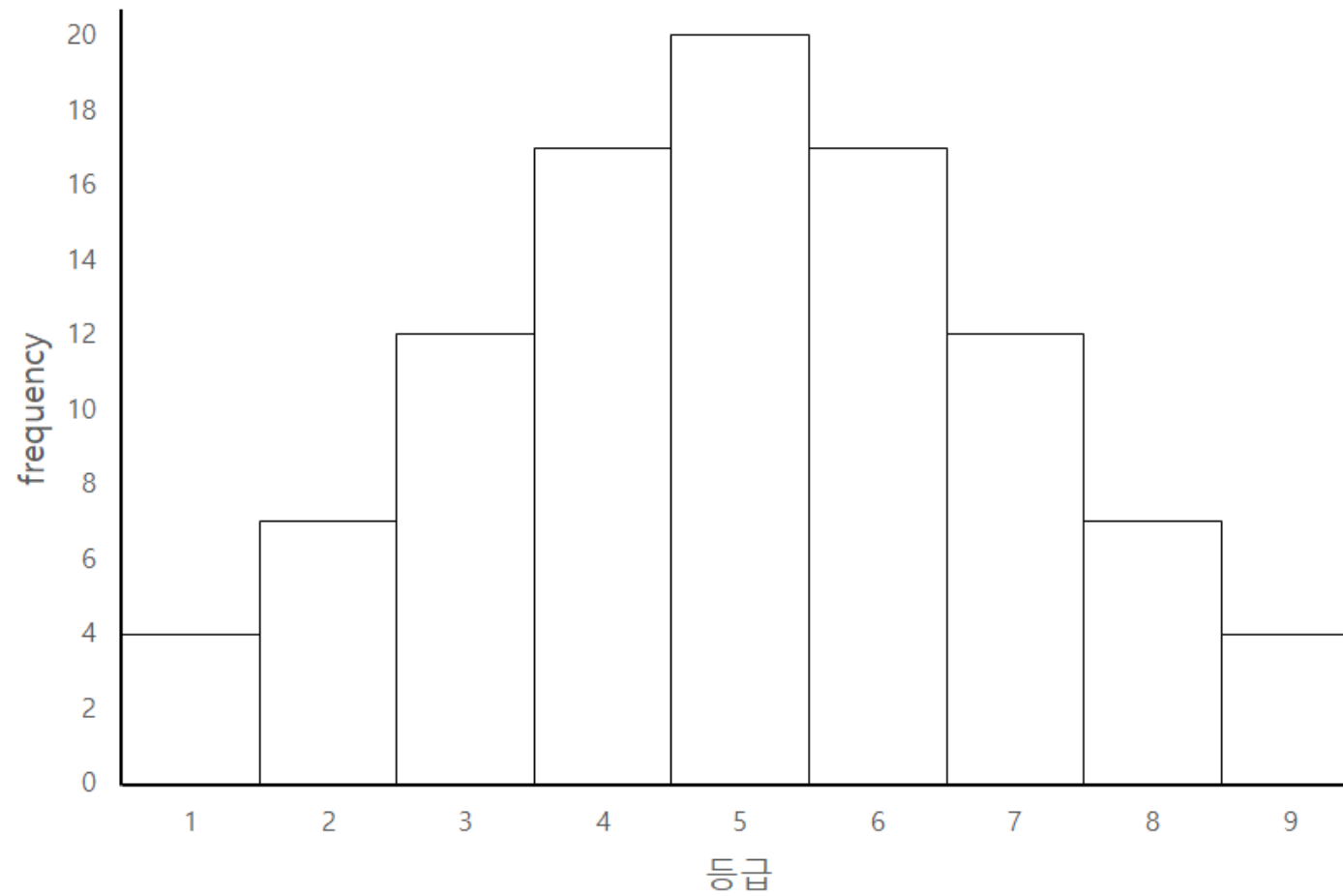


로그-정규 분포

- ▶ 로그를 취하고 나면 정규 분포를 이루는 경우

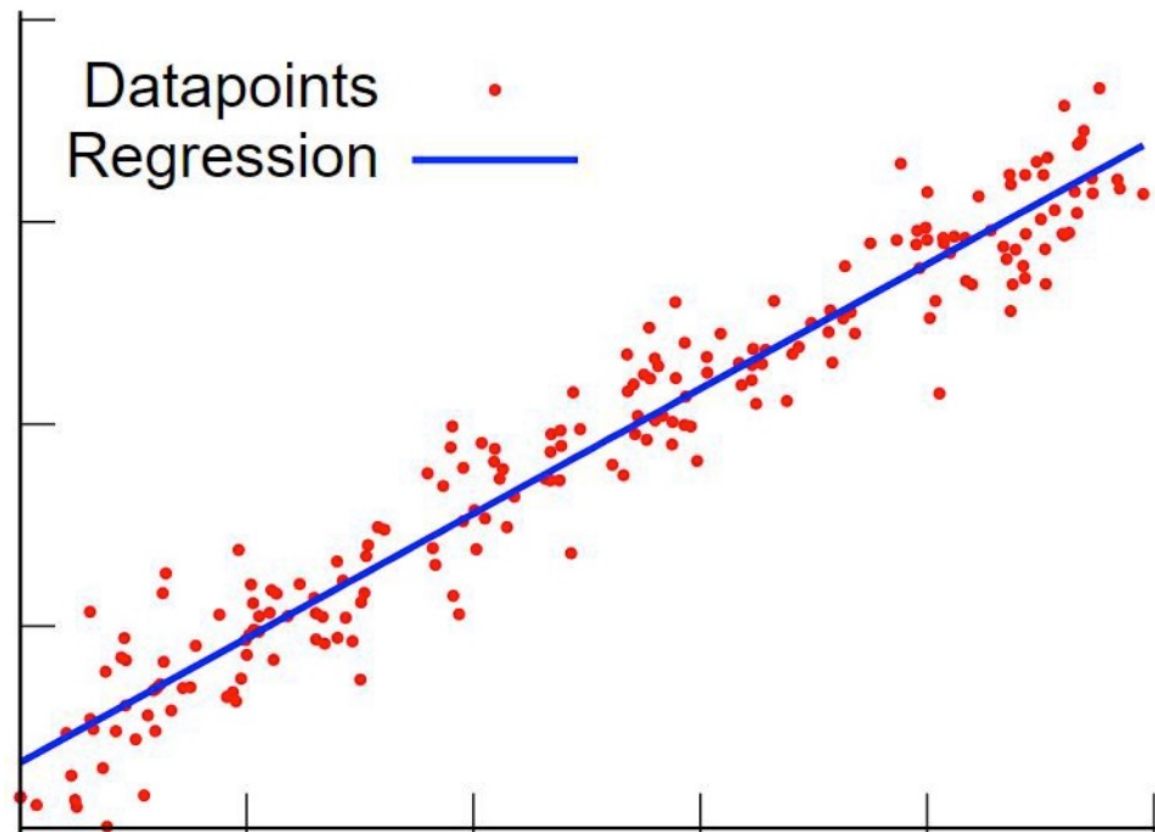


범주형 변환 예



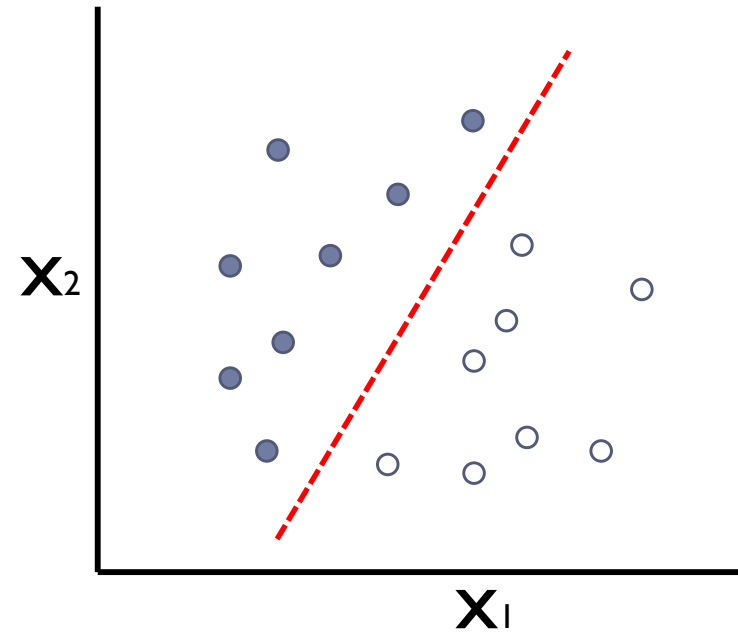
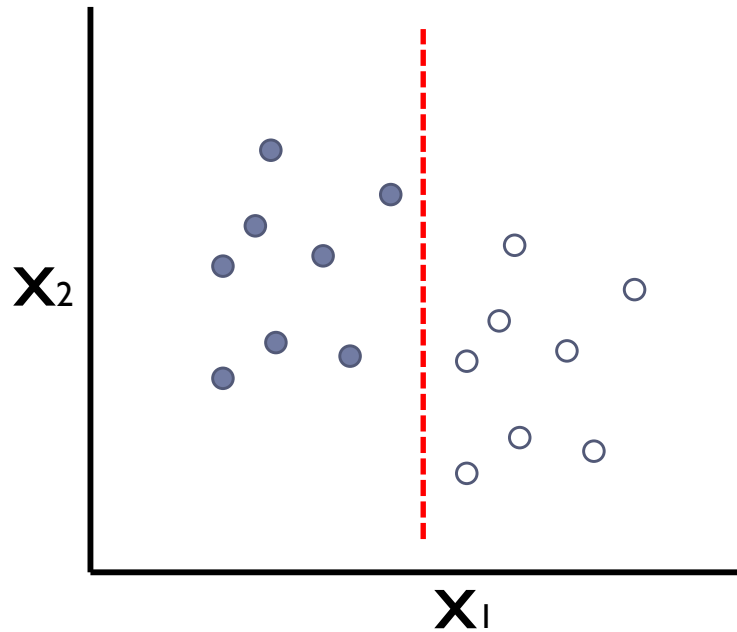
선형 회귀

- ▶ 선형 회귀(regression) $y = wX + b$



선형 분류

- 하나의 변수만 사용하는 경우 선형분류 결정 경계: $x_1 > b$
- 두 개의 변수를 사용하는 경우 선형분류 결정 경계: $a_1 x_1 + a_2 x_2 + b > 0$



손실함수와 성능지표

손실함수와 성능지표

- ▶ **손실함수** (loss function)는 모델을 훈련시킬 때 기준으로 삼기 위해서이다.
 - ▶ 모델은 손실함수를 최소화 하는 방향으로 학습한다
- ▶ **모델의 성능지표**는 이렇게 만든 모델이 궁극적으로 얼마나 잘 동작하는지를 평가하는 척도이다

회귀모델의 손실함수

- ▶ 회귀분석에서는 오차 자승의 합의 평균치(MSE: mean square error)를 손실함수로 주로 사용한다

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- ▶ N: 배치 크기
- ▶ 배치 크기, 학습률과 같은 설정 변수를 하이퍼파라미터라고 한다
 - ▶ **하이퍼파라미터**는 사람이 선택하는 값이며, 머신러닝 모델이 최적화해야 하는 계수는 "**파라미터**"이다

회귀 성능 평가 지표

- ▶ 회귀모델의 성능 평가에는 R-squared가 사용된다.
- ▶ R-Squared,

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

- ▶ 위에서 y_i 는 실제 값, $\hat{y}_i$ 는 예측 값, $\bar{y}$ 는 샘플의 평균 값을 나타낸다
- ▶ 완벽한 회귀 예측 모델은 R^2 값이 1이 된다

혼돈 매트릭스

- ▶ 혼돈 매트릭스(confusion matrix)란 분류의 결과가 잘 맞았는지를 평가하는 채점표와 같다
- ▶ 이진 분류의 경우 네 가지 경우의 수가 존재한다.

실제 \ 예측	p로 예측	n으로 예측
실제로는 p	True positive (TP)	False negative (FN)
실제로는 n	False positive (FP)	True negative (TN)

실제 \ 예측	p로 예측	n으로 예측
실제로는 p	7	3
실제로는 n	10	80

- ▶ $\text{accuracy} = (6+80)/100=0.86$, $\text{recall}=(7/10)=0.7$, $\text{precision}=(7/17)=$
F1-score: recall과 precision의 조화 평균

분류의 성능지표

- ▶ 분류에서는 정확도(accuracy)를 기본으로 측정하나 필요에 따라 다음과 같은 값을 추가로 측정해야 한다
- ▶ 컨퓨전 매트릭스를 사용한다
 - ▶ 정밀도 (precision)
 - ▶ 재현률 (recall)
 - ▶ F-1 점수
 - ▶ ROC
 - ▶ AUC

정확도와 정밀도

▶ 정확도(accuracy)

- ▶ 정확도란 positive 및 negative 두가지에 대해서 전체적으로 정확히 예측한 비율을 말한다.

$$\begin{aligned}\text{정확도} &= (\text{positive 또는 negative를 맞춘 수}) / (\text{전체 샘플 수 } N) \\ &= (TP+TF) / (TP + FP + FN + TN)\end{aligned}$$

▶ 정밀도(precision)

- ▶ 정밀도란 머신러닝 모델이 positive라고 예측한 것 중에 positive인 비율을 말한다.

$$\begin{aligned}\text{정밀도} &= (\text{positive를 맞춘 수}) / (\text{positive라고 예측한 수}) \\ &= TP/(TP+FP)\end{aligned}$$

재현율과 F1 점수

▶ 재현율(recall)

- ▶ 재현율은 전체 positive 샘플 중에 모델이 positive라고 찾아낸 비율을 말한다. 재현율은 관심 있는 대상을 얼마나 찾아내는지 나타낸다.

$$\begin{aligned}\text{재현율} &= (\text{positive를 맞춘 수}) / (\text{positive 전체 수}) \\ &= TP / (TP + FN)\end{aligned}$$

▶ F1 지표

- ▶ 재현율과 정밀도 두 성능평가 지표를 동시에 측정하는 방법
- ▶ 정밀도와 재현율의 조화평균(harmonic average)으로 정의한다.

$$F1 = 2 \times \text{precision} \times \text{recall} / (\text{precision} + \text{recall})$$

조화 평균

- ▶ 산술 평균과 달리 점수들의 차이가 크면 산술평균보다 더 나은 점수를 얻는 방식
 - ▶ 점수들이 조화롭지 않으면(비슷하지 않으면) 페널티를 더 주는 방식

$$\text{조화 평균: } \frac{1}{c} = \frac{\left(\frac{1}{a} + \frac{1}{b}\right)}{2}$$

$$c = \frac{2ab}{a+b}$$

정보량과 엔트로피

- ▶ 정보량은 일어날 확률의 역수에 비례한다고 가정

- ▶ 정보량 = $\log\left(\frac{1}{p}\right)$

- ▶ 엔트로피: 정보량의 기대치로, 어떤 사건이 갖는 가치와 그 사건이 발생할 확률의 곱

$$p \log\left(\frac{1}{p}\right) = -p \log(p)$$

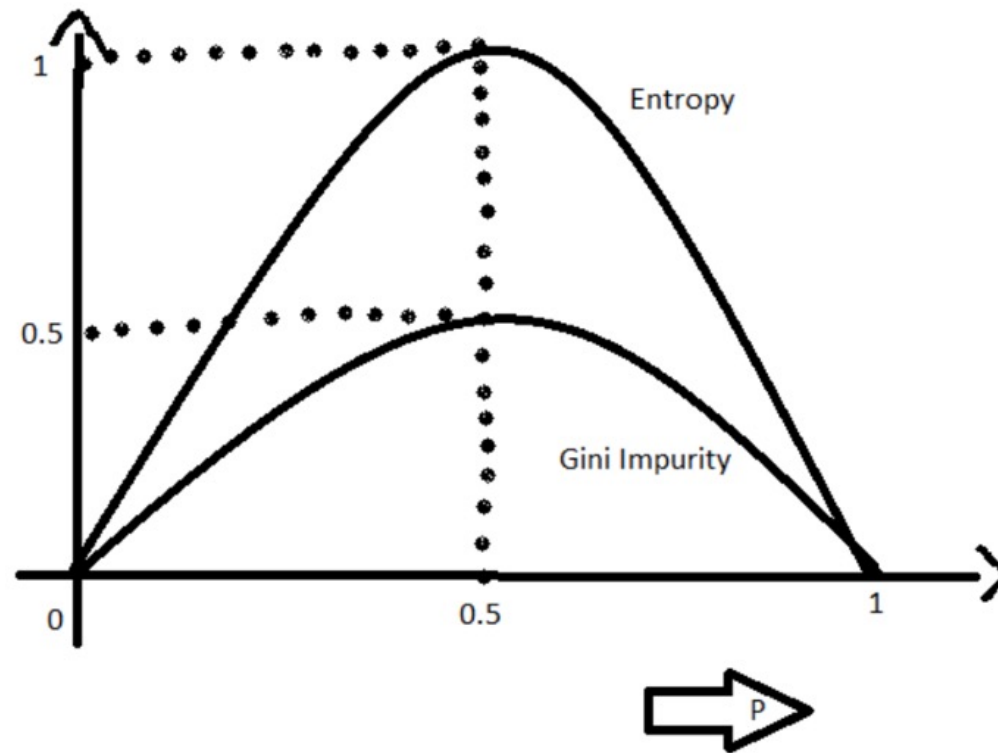
- ▶ 분류시 순도(purity)를 측정하는데 사용된다
 - ▶ 한 노드에 여러 클래스가 골고루 균일하게 섞여 있을 때는 엔트로피가 가장 높고, 동종의 클래스로 모여 있을수록 엔트로피가 낮다

분류의 손실함수

- ▶ 분류에서 예를 들어 정확도(accuracy)를 손실함수로 사용하면 다음과 같은 문제가 있다
 - ▶ **카테고리 분포 불균형**시 문제
 - ▶ 클래스 발생빈도가 다른 경우, 적게 발생하는 클래스를 잘 찾지 못한다
- ▶ 분류 모델에서는 이를 보완하기 위해서 크로스 엔트로피(cross entropy)를 손실함수로 사용한다
 - ▶ m: 클래스 수

$$L = -\frac{1}{m} \sum_{i=1}^m y_i \cdot \log(\hat{y}_i)$$

엔트로피와 지니계수



$$E(S) = \sum_{i=1}^c -p_i \log_2 p_i$$

$$Gini(E) = 1 - \sum_{j=1}^c p_j^2$$

손실함수와 성능지표

- ▶ 대표적인 손실함수와 성능 평가 지표
 - ▶ 분류에서는 정확도 외에 성능 지표로 정밀도(precision), 재현률(recall), F1-지수 등이 추가로 사용된다.

	손실함수	성능평가지표
정 의	손실함수를 줄이는 방향으로 모델이 학습을 함	성능을 높이는 것이 머신러닝을 사용하는 목적임
회귀 모델의 대표적인 값	MSE (오차 자승의 편균치)	R^2
분류 모델의 대표적인 값	크로스 엔트로피	정확도, 정밀도, 재현률, F1점수

과대적합

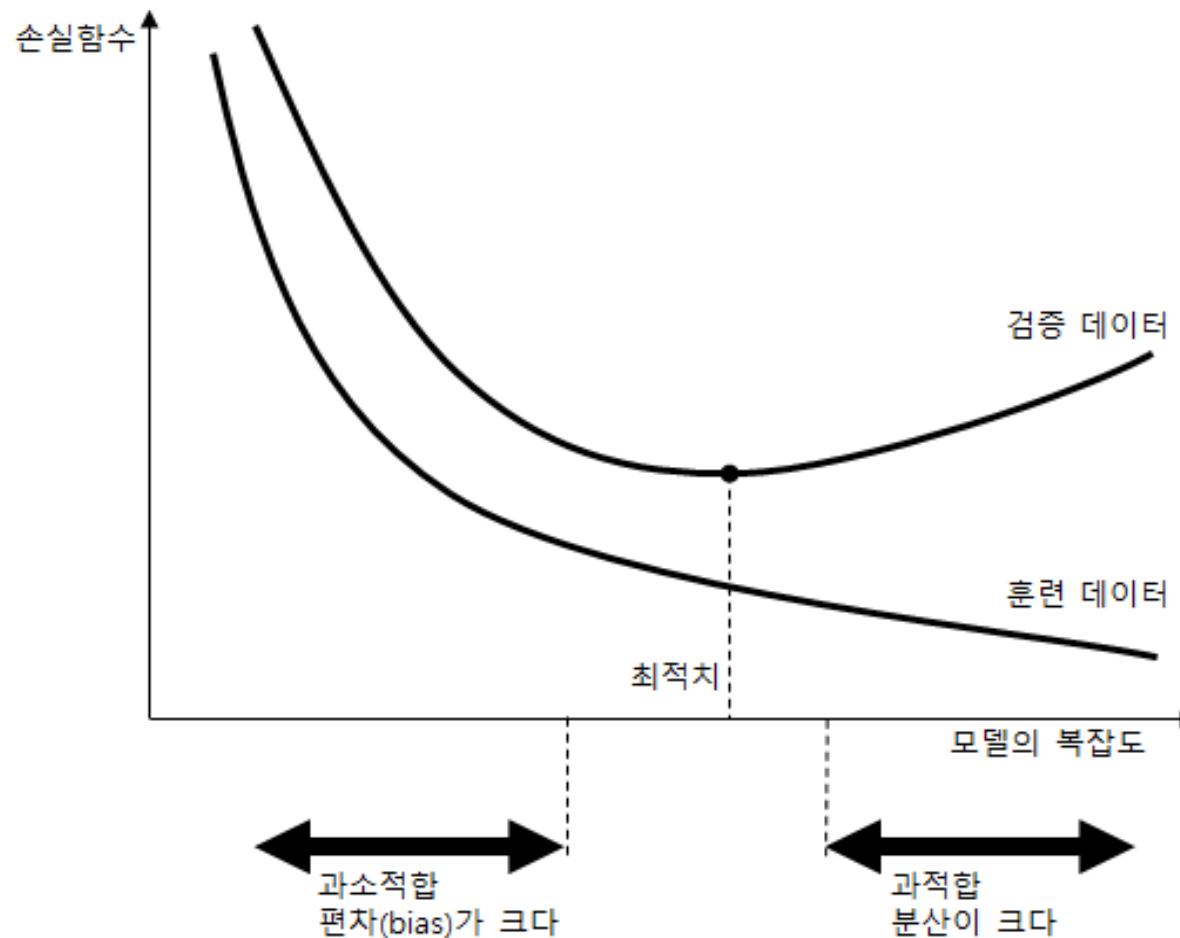
- ▶ 모델이 훈련 데이터에 대해서만 잘 동작하도록 만들어져서 새로운 데이터에 대해서는 오히려 잘 동작하지 못하는 경우를 과대적합(over fitting)되었다고 한다
- ▶ 과대적합은 주어진 훈련 데이터를 너무 세밀하게 학습에 반영하여 발생하는 현상이다
- ▶ 머신러닝에서는 과대적합을 피해서 일반적으로 잘 동작하게 모델을 만드는 것이 매우 중요하다
 - ▶ 이를 모델의 일반화(generalization)라고 한다
- ▶ 일반화 방법
 - ▶ 모델을 단순하게 만든다(이를 규제화(regularization)라고도 한다)
 - ▶ 더 많은 훈련 데이터를 확보한다

과소적합

- ▶ 과대적합과 반대로 모델이 너무 간단하거나 학습이 덜 되어 성능이 미흡한 경우를 과소적합(under fitting)이라고 한다
- ▶ 과소적합을 피하려면
 - ▶ 좀 더 상세한 모델 구조를 사용한다
 - ▶ 학습을 더 오래 수행한다 (같은 데이터를 반복하여 학습에 사용)
- ▶ 머신러닝에서는 과대적합과 과소적합을 모두 피해야 한다

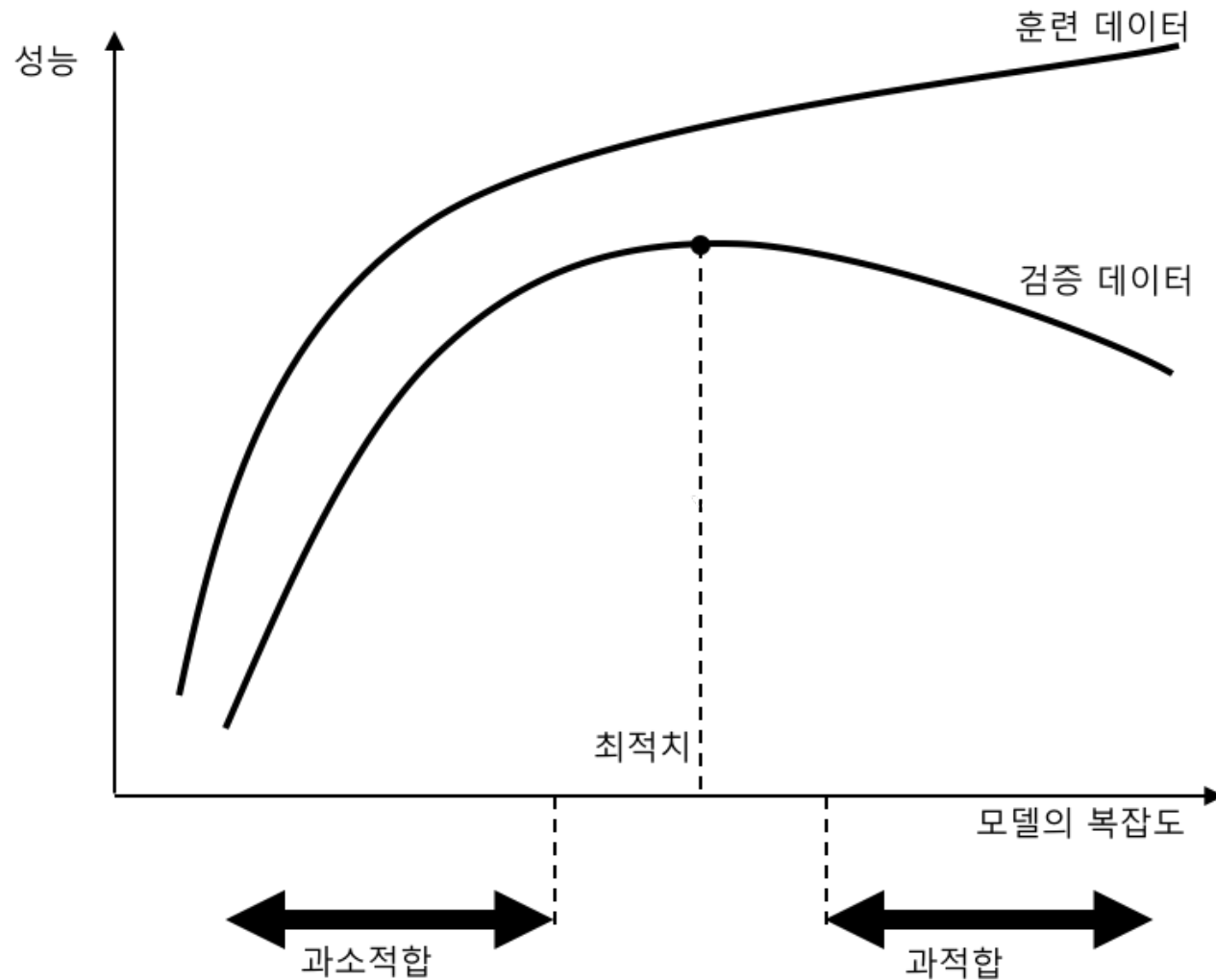
과대적합 검증

- ▶ 훈련 데이터에 대한 성능과 검증 데이터에 대한 손실함수를 관찰한다

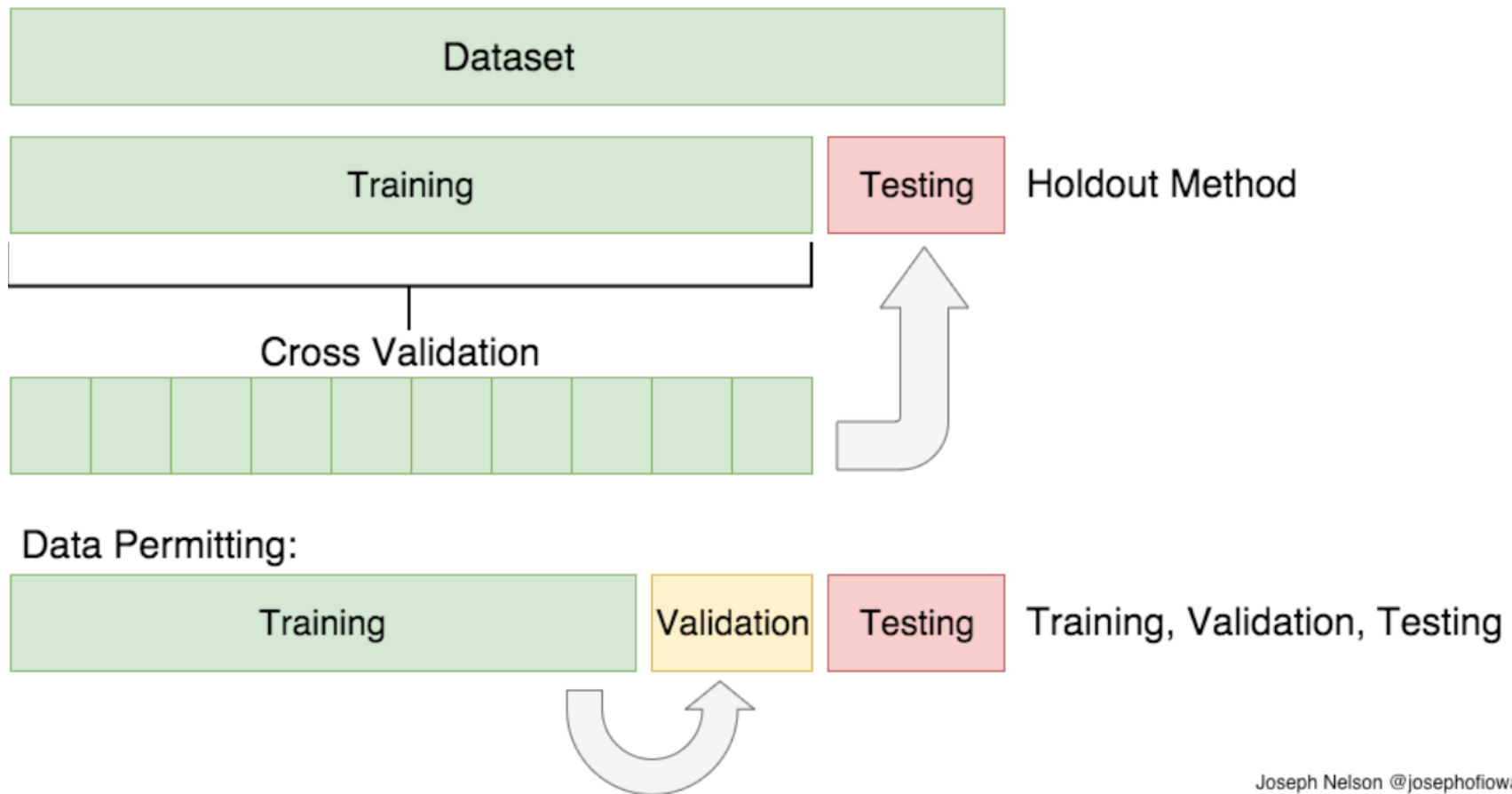


과대적합 검증

▶ 성능 비교



훈련, 검증, 테스트 데이터



Joseph Nelson @josephoflow

교차 검증

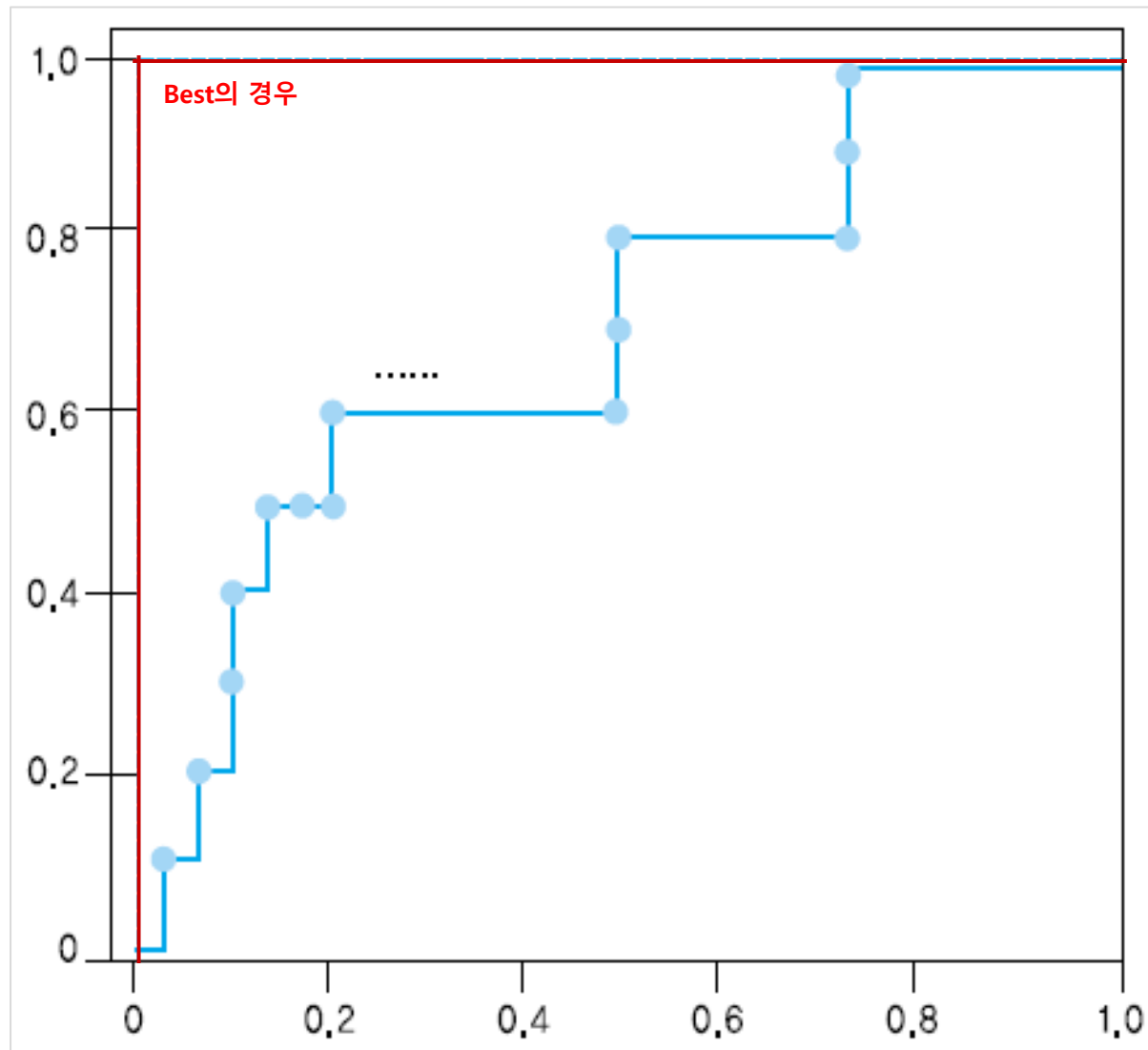
- ▶ cross validation
- ▶ 주어진 데이터를 훈련 및 테스트 데이터로 한번만 나누어 사용하지 않고 여러가지 경우로 나누어 사용하는 방식
- ▶ K-fold 교차 검증이 널리 사용된다



순위 평가

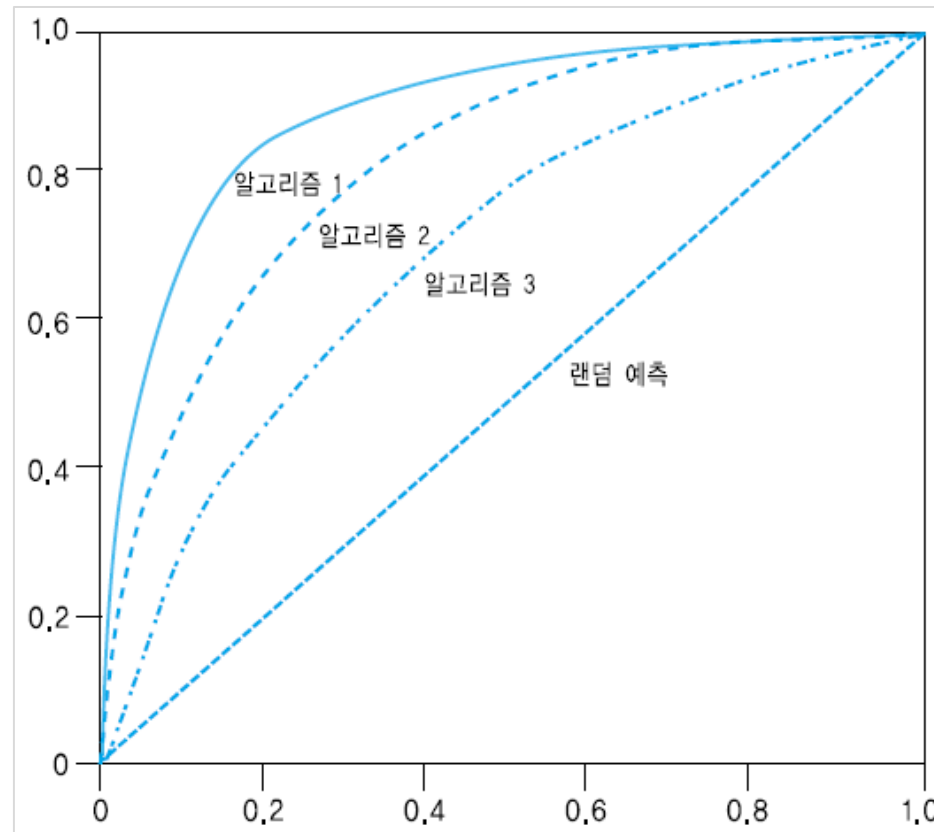
- ▶ 분류를 예측한 순서를 고려한 평가 방식
 - ▶ 찾고자 하는 positive 샘플에게 높은 점수를 주었는지를 평가하는 것
- ▶ ROC 커브
 - ▶ Receiver Operating Characteristic
 - ▶ 예측 결과를 점수가 높은 순서대로 예측한 것이 실제 값과 얼마나 순서에 따라 잘 맞는지는 검증하는 2차원 그래프
 - ▶ (0,0)점에서 시작하여 정답을 맞추었으면 y축 위로 한 칸 이동, 정답을 맞추지 못했으면 x축 방향으로 한 칸 이동하며 종점은 (1, 1)이다
 - ▶ y축은 예측이 맞은 것을 나타내므로 true positive rate, x축은 예측이 틀린 것을 나타내므로 false positive rate라고 한다
- ▶ AUC(Area Under the Curve)
 - ▶ ROC 성능을 간단히 하나의 수치로 나타내기 위해서 ROC 그래프의 면적을 사용하는 것

ROC 그래프 예시



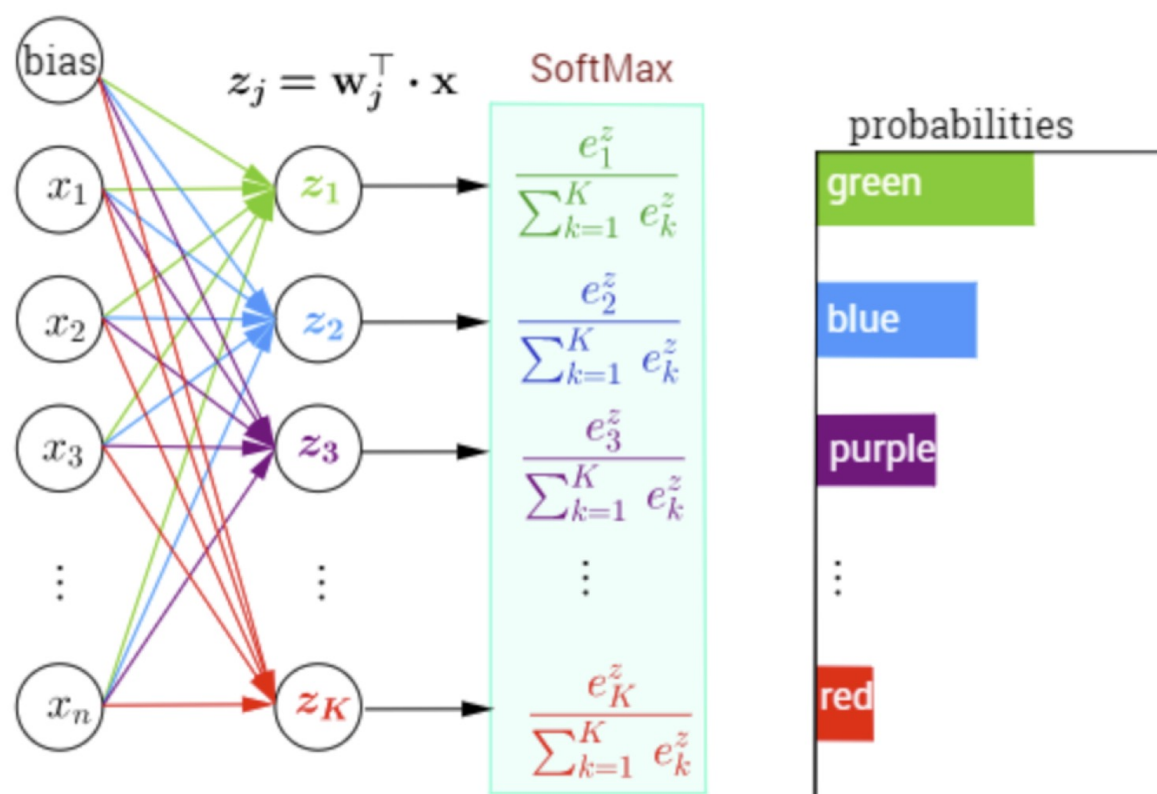
AUC(Area Under Curve)

- ▶ ROC 그래프의 면적을 계산
- ▶ 우수한 알고리즘일수록 초반에 y 축 상단 방향으로 이동하므로 ROC 커브의 면적이 넓어짐



소프트맥스

$$\hat{p}_k = \sigma(s(x))_k = \frac{\exp(s_k(x))}{\sum_{j=1}^K \exp(s_j(x))}$$

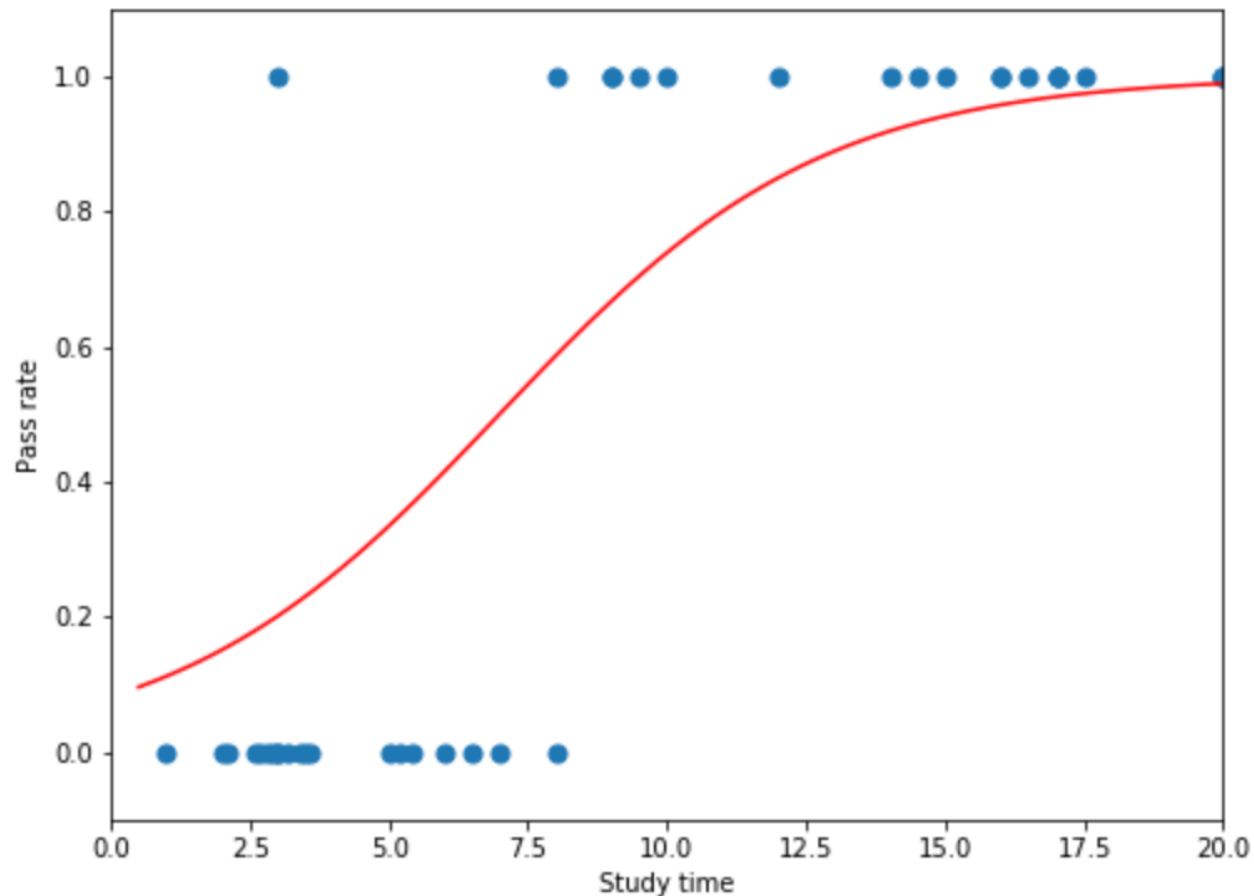


소프트맥스

- ▶ 이진 분류가 아니라 3개 이상의 클래스 중에 하나를 예측해야 하는 경우는 로지스틱 회귀를 사용할 수 없다
- ▶ 이때는 소프트맥스 (softmax) 함수를 이용한다
 - ▶ 각 클래스로 분류될 가능성을 나타내는 점수를 먼저 구하고 이 점수들을 사용하여 상대적인 확률의 비율을 구한다

로지스틱 회귀

- ▶ 로지스틱 회귀분석(logistic regression)은 임의의 범위를 갖는 선형적 값으로부터 이진 분류의 확률을 예측하는 모델

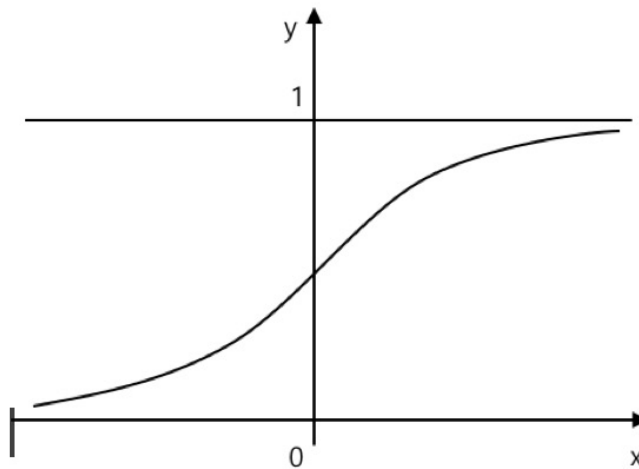


sigmoid

- ▶ 입출력의 관계를 S형 커브를 사용하여 선형값을 확률로 매핑할 때 사용되는 함수
- ▶ S 커브를 시그모이드 함수라고 한다

$$p = \frac{1}{1 + e^{-y}}$$

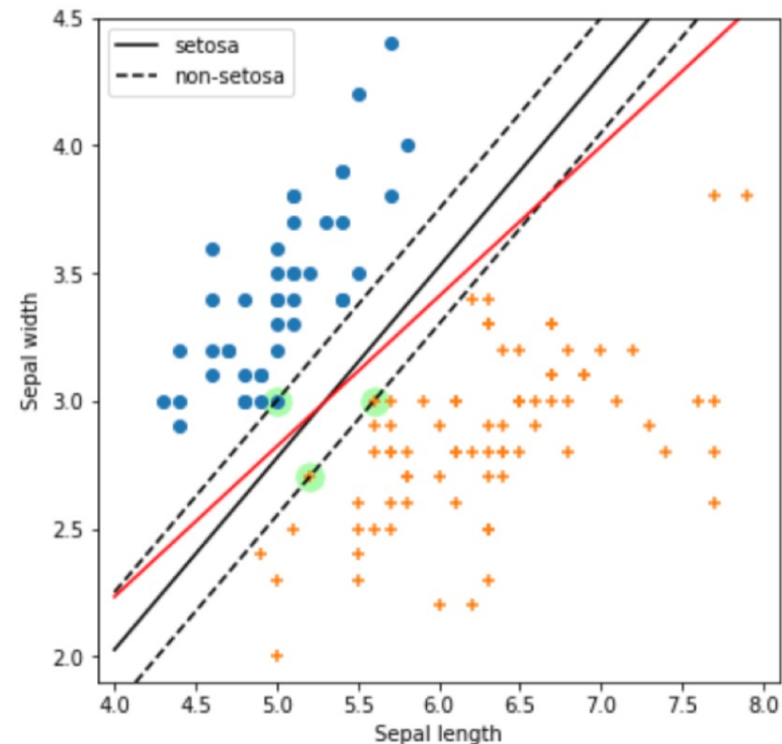
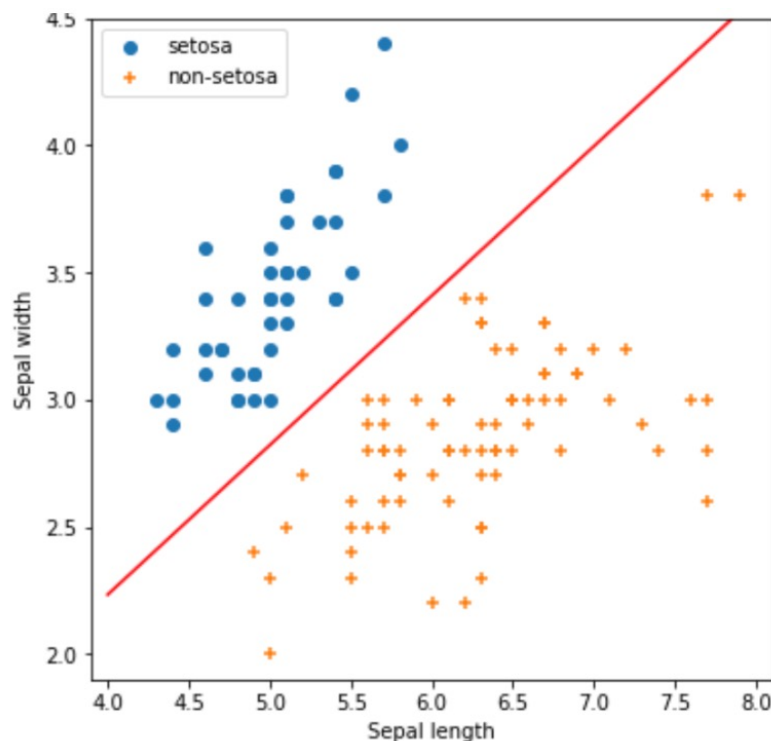
$$p = \frac{1}{1 + e^{-(ax+b)}}$$



SVM

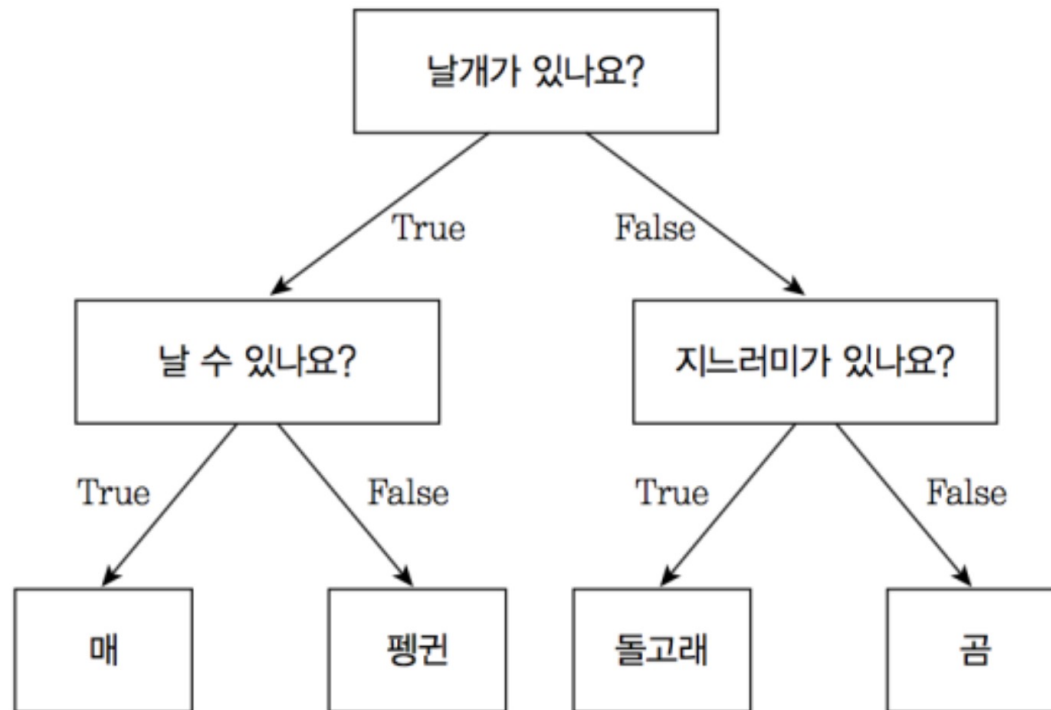
SVM

- ▶ 서포트 벡터 머신(Support Vector Machine, SVM)은 선형 모델을 개선한 것
- ▶ 기본 아이디어는 분류시에 경계면을 가능한 일반화 한 것
 - ▶ 분류시 샘플 집단 간의 거리를 가능한 멀리 나눌 수 있는 경계면을 찾는다 (Margin을 최대화 하는 경계면을 찾는 것)



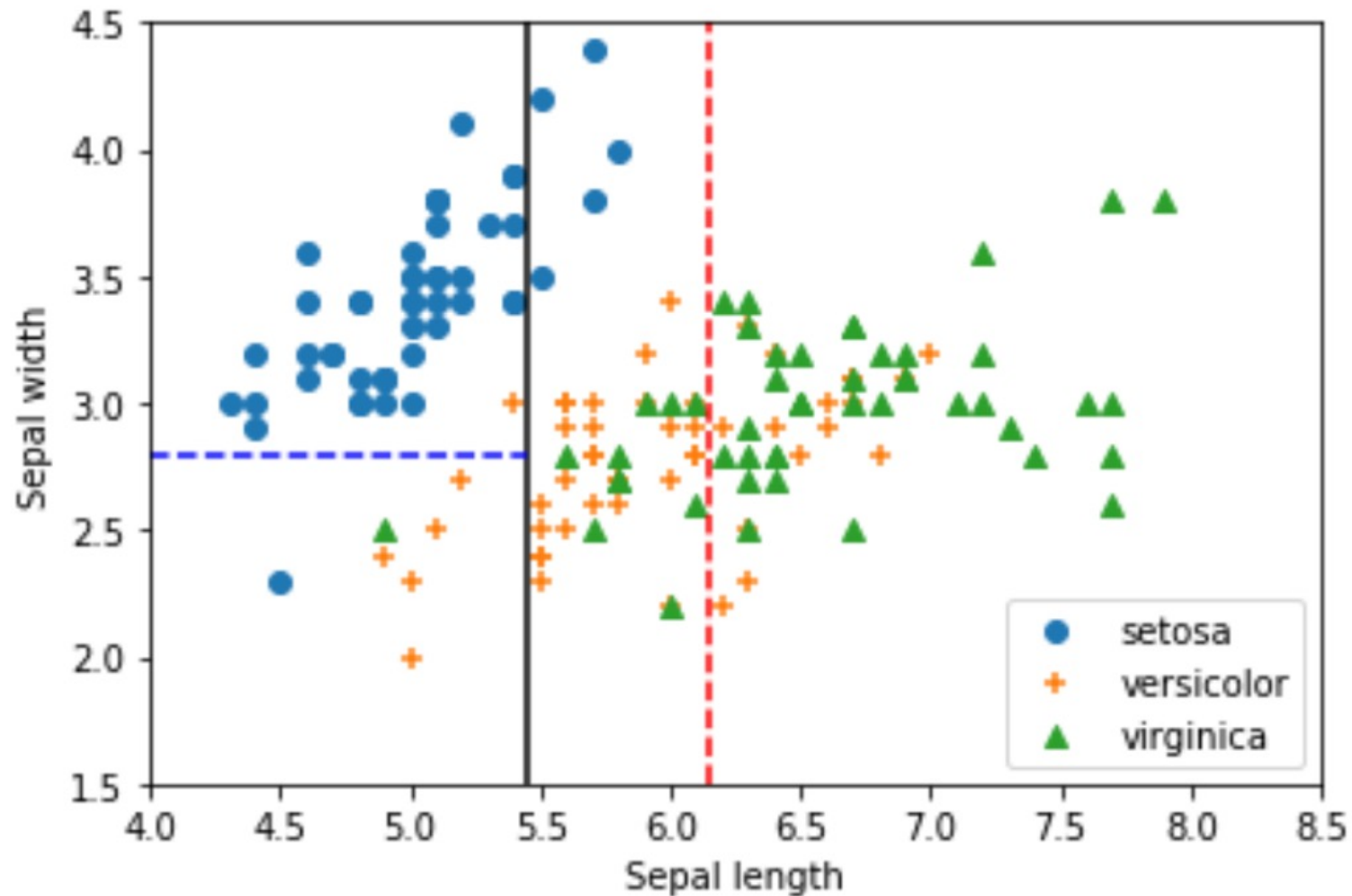
결정트리

- ▶ 분류 모델
 - ▶ 하위 그룹에 가능하면 같은 종류의 샘플이 모이도록 한다
- ▶ 회귀 모델
 - ▶ 하위 그룹의 샘플의 분산(variance)이 적도록 나눈다



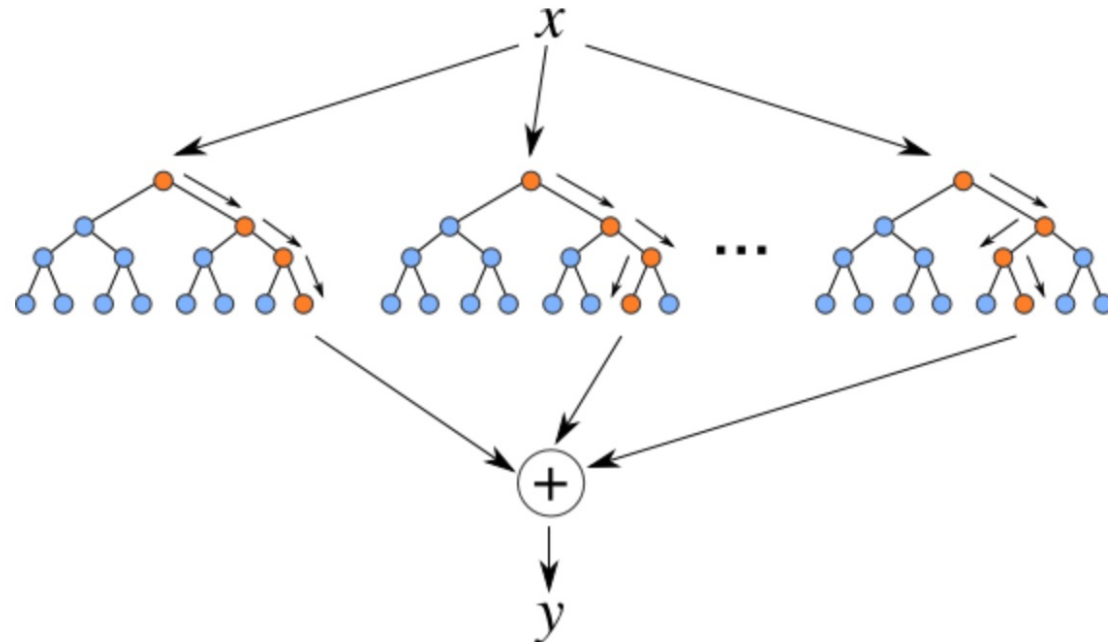
결정트리

- ▶ 한번에 한 특성씩을 대상으로 훈련 샘플을 (두개의) 그룹으로 나누되 각 그룹에 가능한 유사한 샘플들이 모이도록 한다



랜덤 포레스트

- ▶ Random Forest, 결정 트리의 성능을 개선한 방법
- ▶ 간단한 구조의 결정 트리(weak learner)를 수백개 만들고 각 결정 트리의 동작 결과의 평균치를 사용하는 방법
 - ▶ 랜덤 포레스트를 구성하는 약한 결정 트리를 만들 때 훈련 데이터의 일부만 사용하거나 특성의 일부만 무작위로 선택한다
- ▶ 이를 앙상블(ensemble) 방법이라고 하며 결정 트리 모델보다 좋은 성능을 얻는다

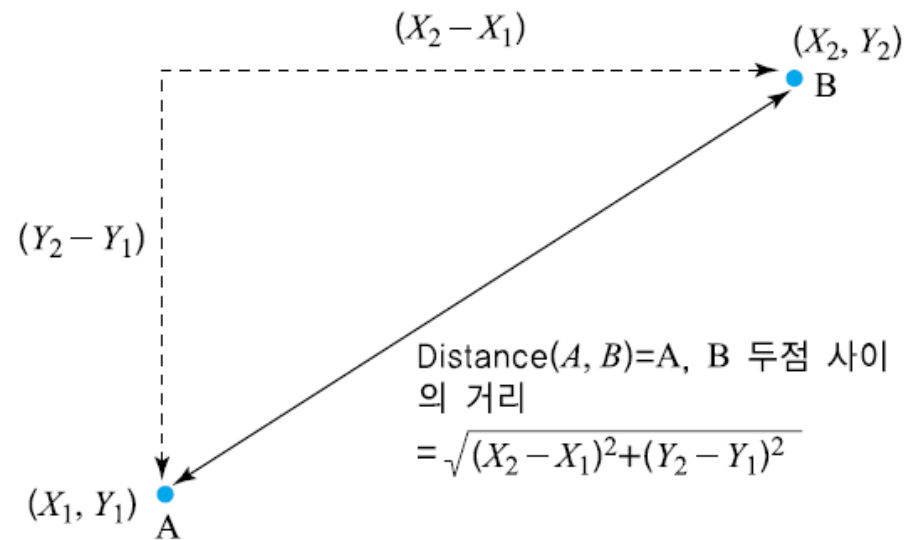


특성 중요도

- ▶ `feature_importances_`
 - ▶ 결정 트리 모델을 만든 후에, 어떤 특성이 결정 트리를 생성할 때 중요한 역할을 했는지 상대적인 비중을 파악할 수 있다
 - ▶ 내부 변수인 `feature_importances_`에서 확인할 수 있다

유클리디언 거리

- ▶ 기하학의 공간(space) 상의 직선 거리
 - ▶ 유클리디언 (Euclidean) 거리

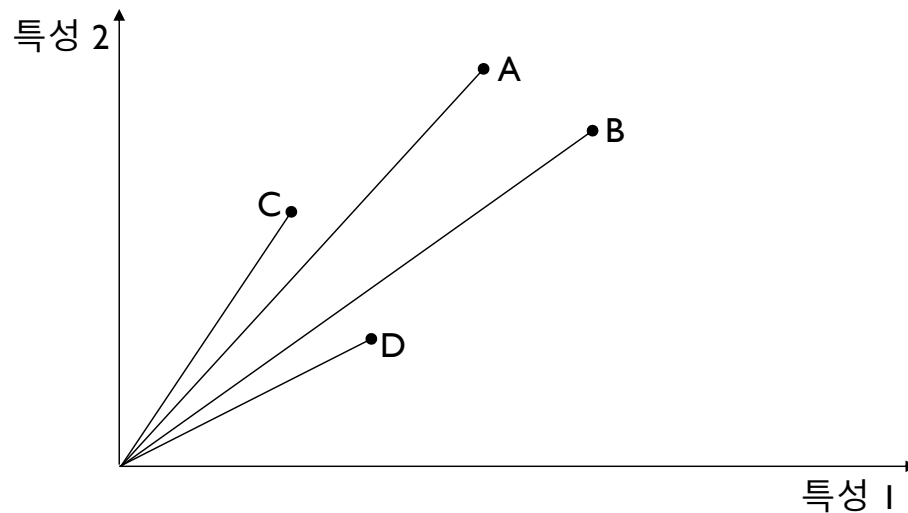


- ▶ n 차원 공간상의 두 점의 거리

$$\sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2 + \dots + (p_n - q_n)^2} = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

코사인(cosine) 유사도

- ▶ 공간상의 두 점이 만드는 각도를 기준으로 유사도를 측정하는 방법 (-1 ~ +1 사이의 값을 갖는다)
- ▶ 공간상 거리가 멀어도 두 점이 가리키는 방향이 같으면 서로 비슷하다고 보는 것



$$s_{\cos}(x, y) = \frac{X \cdot Y}{|X||Y|}$$

- ▶ A와 C가 가깝고 B와 D가 가깝다고 정의

자카드 유사도

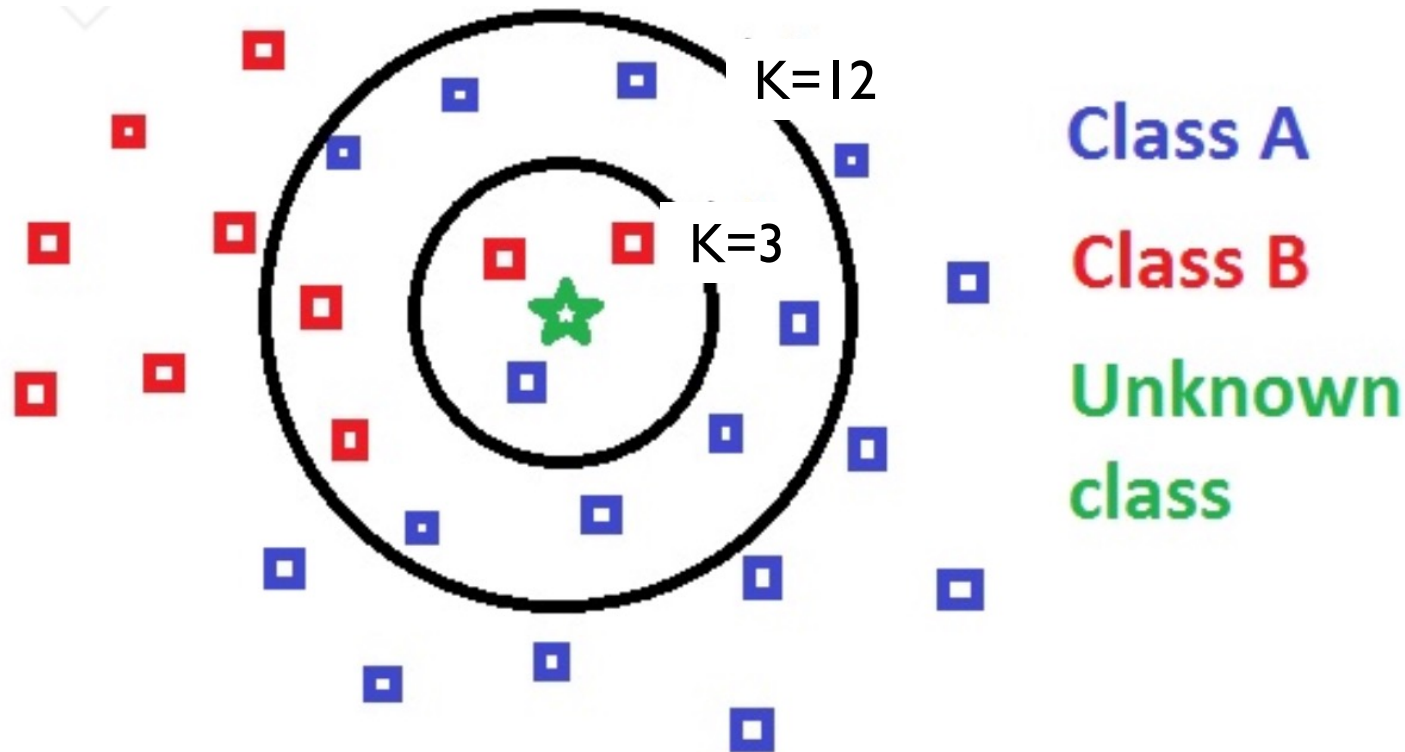
- ▶ 어떤 두 항목이 겹치는 부분의 절대량만을 보지 않고, 두 항목의 공통 부분이 얼마나 많은지를 고려하여 이에 대한 상대적인 값을 유사도로 사용하는 것

$$S_{Jaccard}(X, Y) = \frac{|X \cap Y|}{|X \cup Y|}$$

- ▶ A, B, C가 각각 지난해 본 영화의 총 개수가 20편, 50편, 200편
- ▶ $J(A, B) = 5 / (20 + 50 - 5) = 0.076$
- ▶ $J(A, C) = 10 / (20 + 200 - 10) = 0.047$
- ▶ 즉, $0.076 > 0.047$ 이므로 A와 B가 더 가깝다고 할 수 있음

kNN 하이퍼파라미터

- ▶ k 값이 작을수록 과대적합이 되고 k 가 크면 과소적합이 된다



- ▶ 모델 훈련시간이 필요 없지만 새로운 샘플이 추가되면 상호 거리 계산을 다시 해야 한다

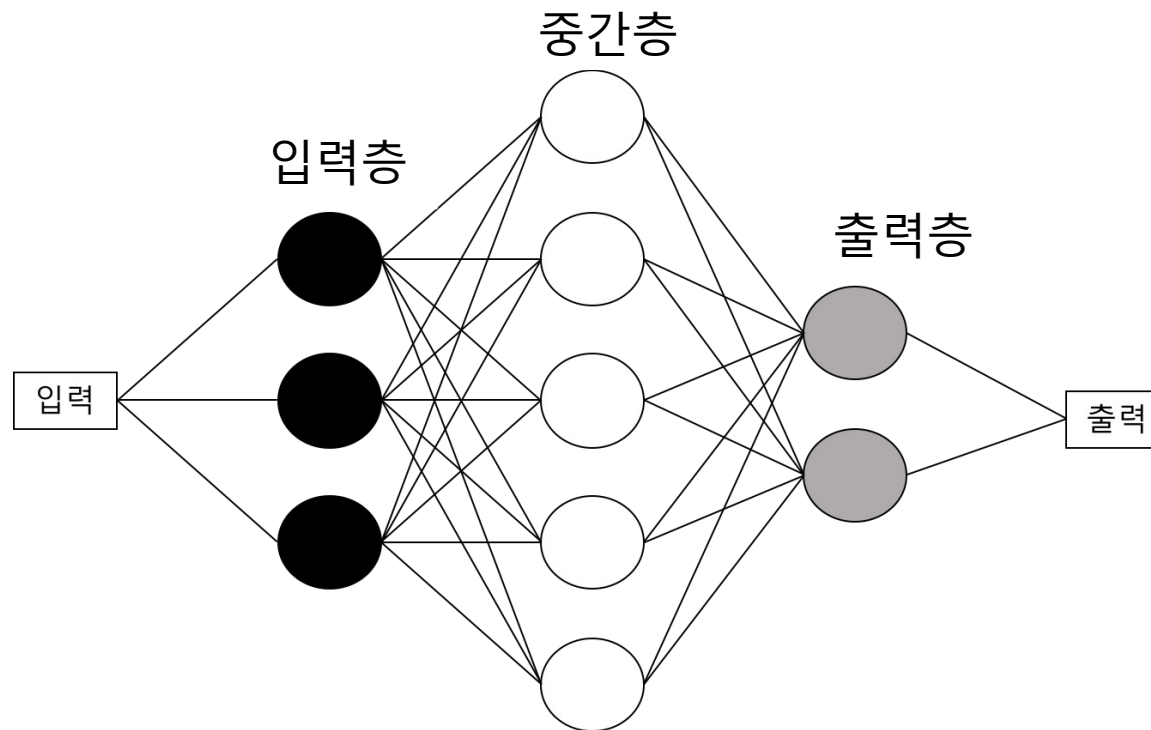
모델 최적화

- ▶ 최적의 머신러닝 모델을 만드는 과정
 - ▶ 모델 구조 선택
 - ▶ 모델 학습 - 파라미터 학습
 - ▶ 모델 최적화 - 최적의 하이퍼 파라미터 선택
- ▶ 하이퍼파라미터의 최적화
 - ▶ 과대적합과 과소적합을 피하면서 성능평가를 최대로 하는 하이퍼 파라미터를 찾는 것

딥러닝

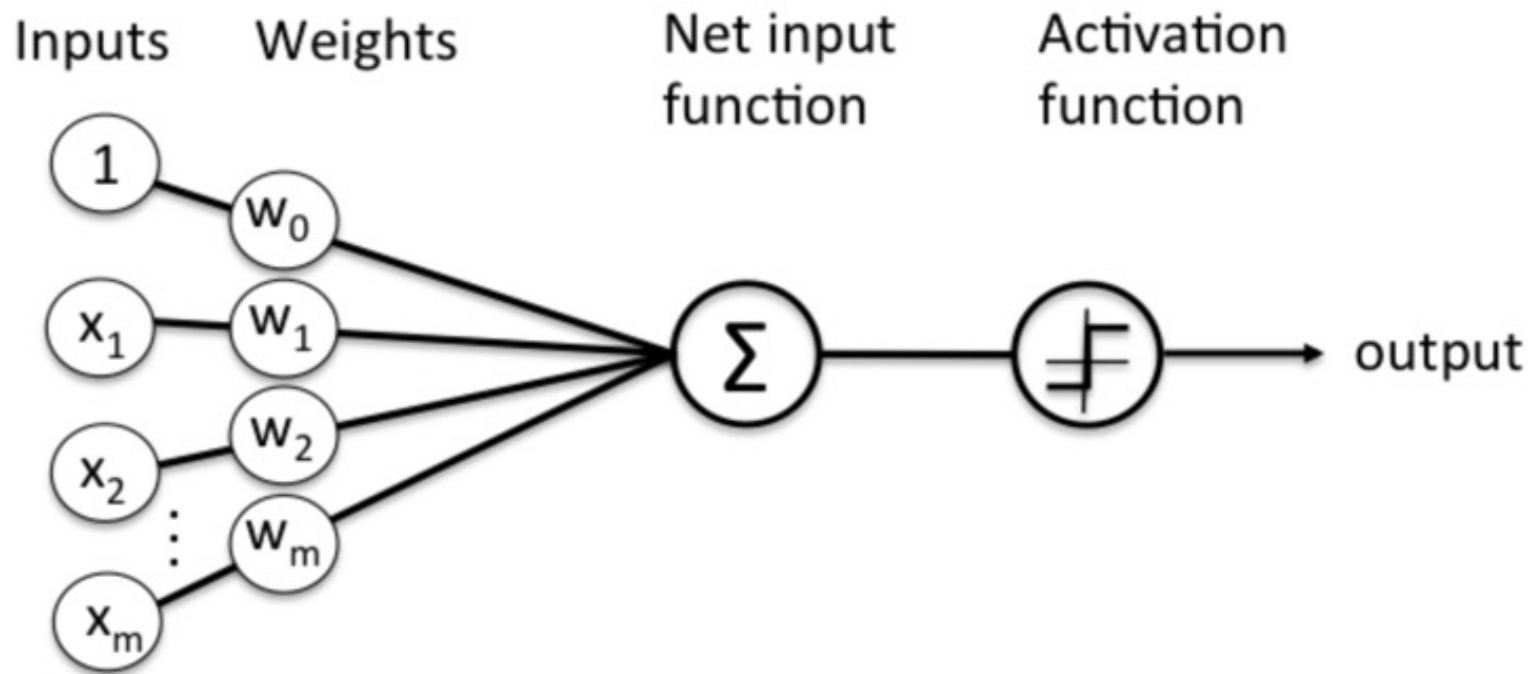
MLP (Multi Layer Perceptron)

- ▶ 입력 데이터에 가중치를 곱하여 중간 출력을 만들고 이 중간 출력에 다시 가중치를 곱하여 다음 계층의 입력으로 사용하는 구조



활성화 함수

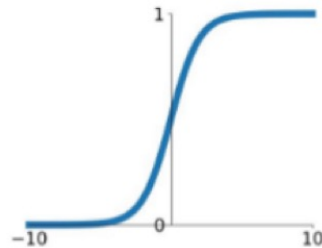
- ▶ MLP 각 계층에서, 가중합을 구한 후에 활성화 함수 (activation function)를 통과시킨다
- ▶ 이는 생물학적인 뉴런의 동작을 모방하여 비선형적인, 메모리 기능을 갖게 한 것



활성화 함수 종류

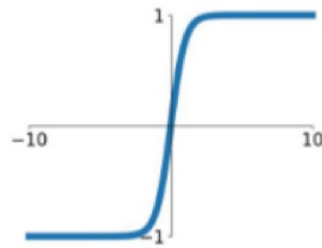
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



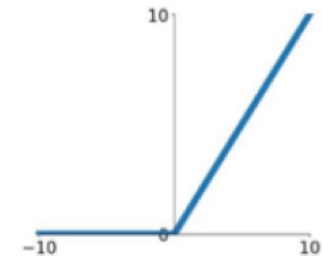
tanh

$$\tanh(x)$$



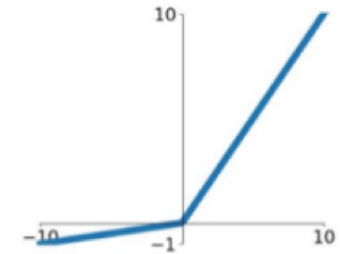
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

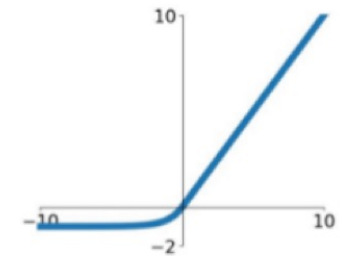


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

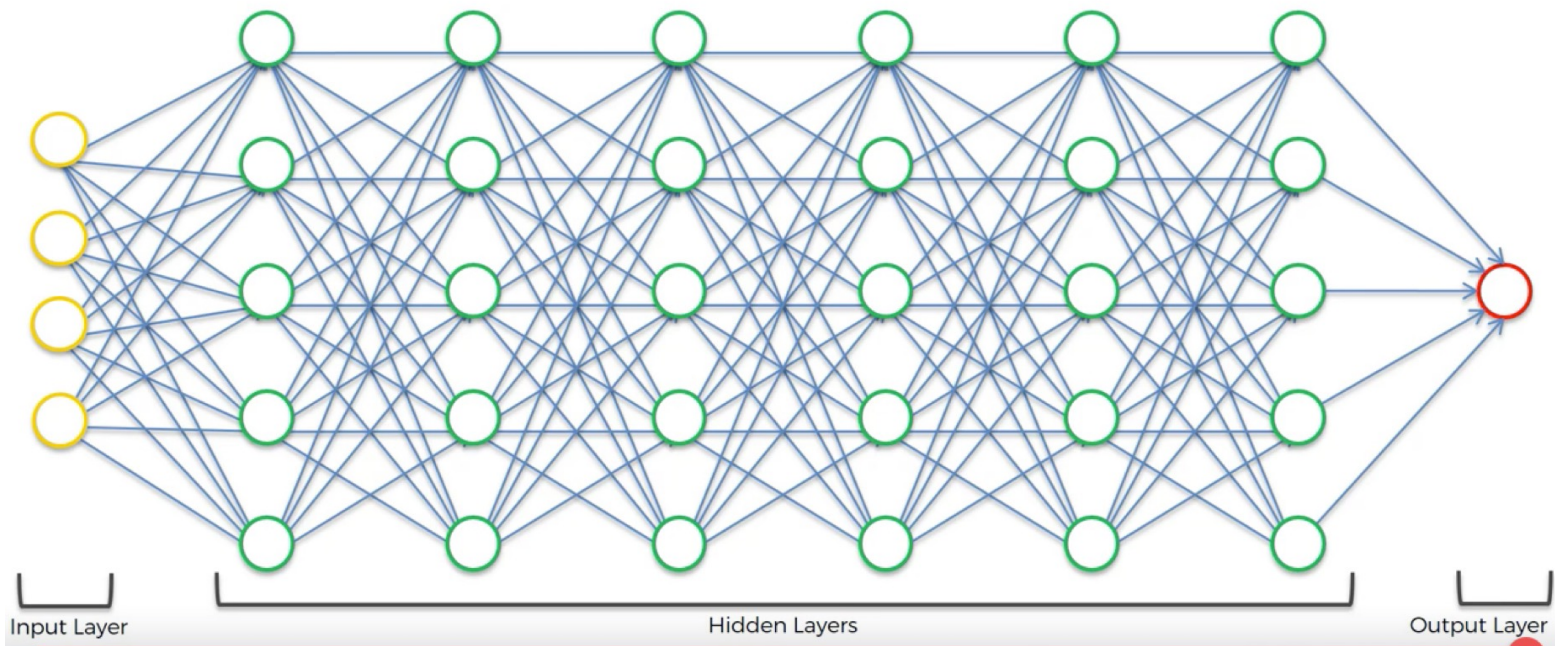
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



MLP 예

- ▶ 히든 계층이 6개인 예 (MLP는 2개 층만 사용해도 동작한다)

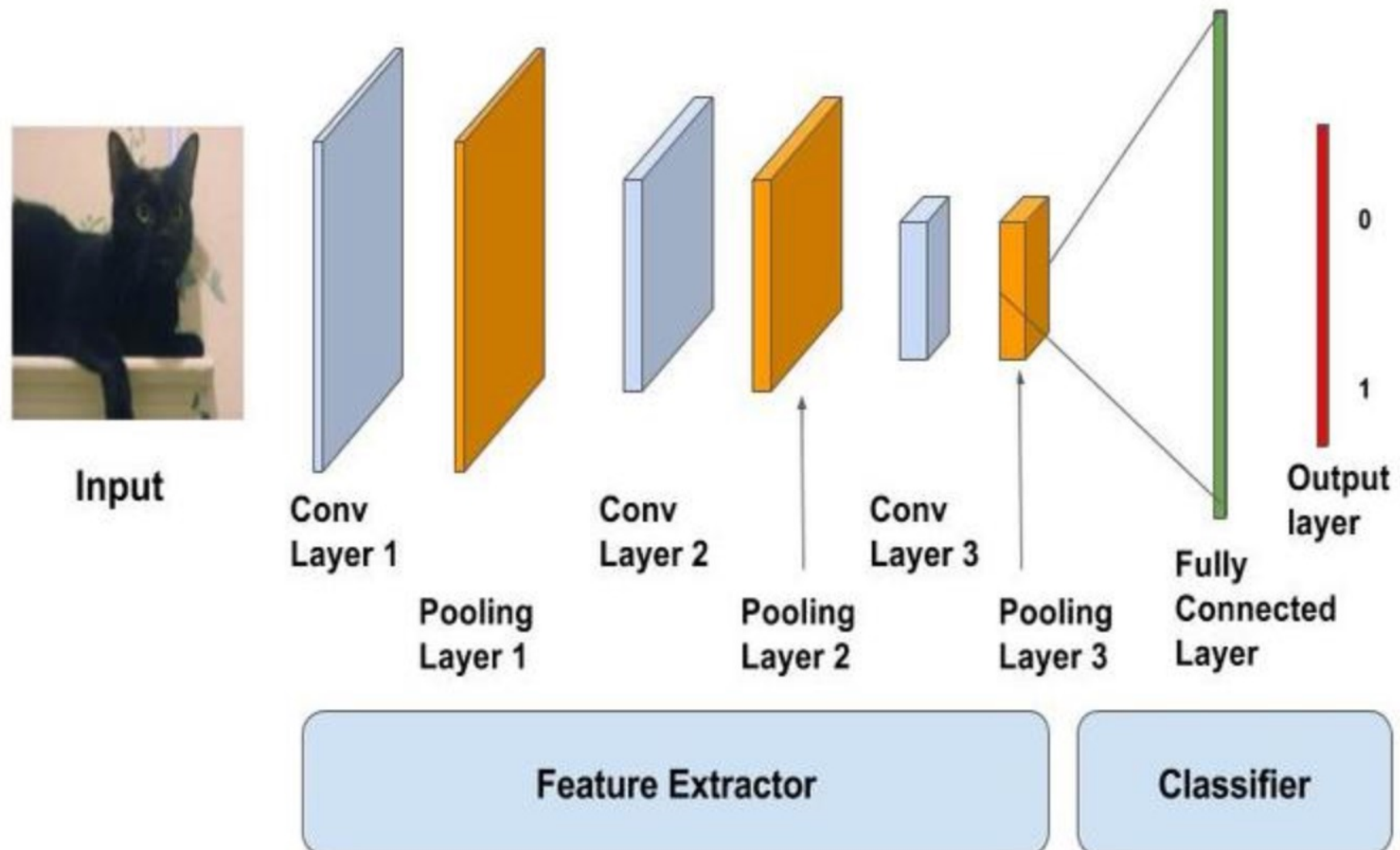


Convolution Neural Network

- ▶ 아래와 같이 임의의 크기와 모양을 가진 이미지 판독에서는 MLP가 동작하지 않는다
 - ▶ MLP에서는 입력신호 전체를 한번에 다룬다
- ▶ 합성곱 신경망 (Convolution Neural Network, CNN)은 이를 개선하였다

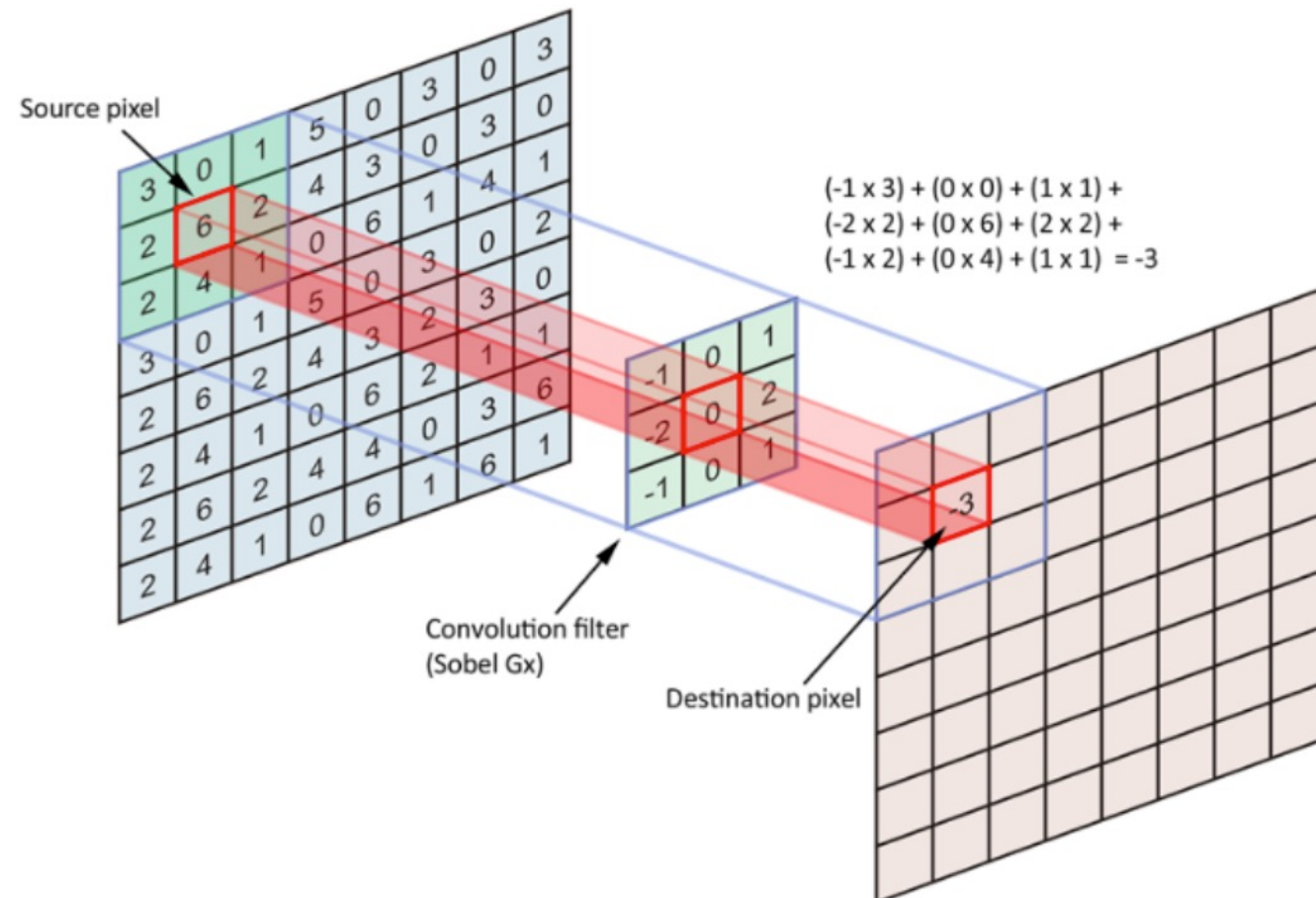


CNN

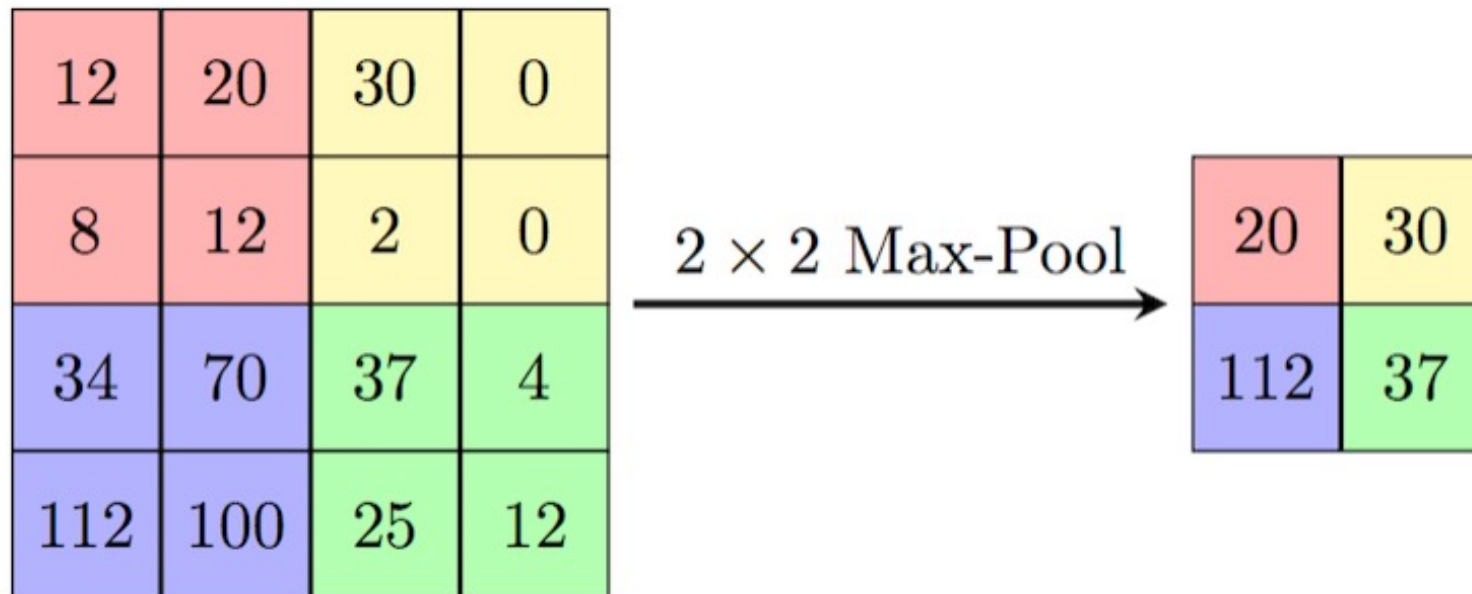


CNN 동작

- ▶ 합성곱 동작
- ▶ 다층 필터링

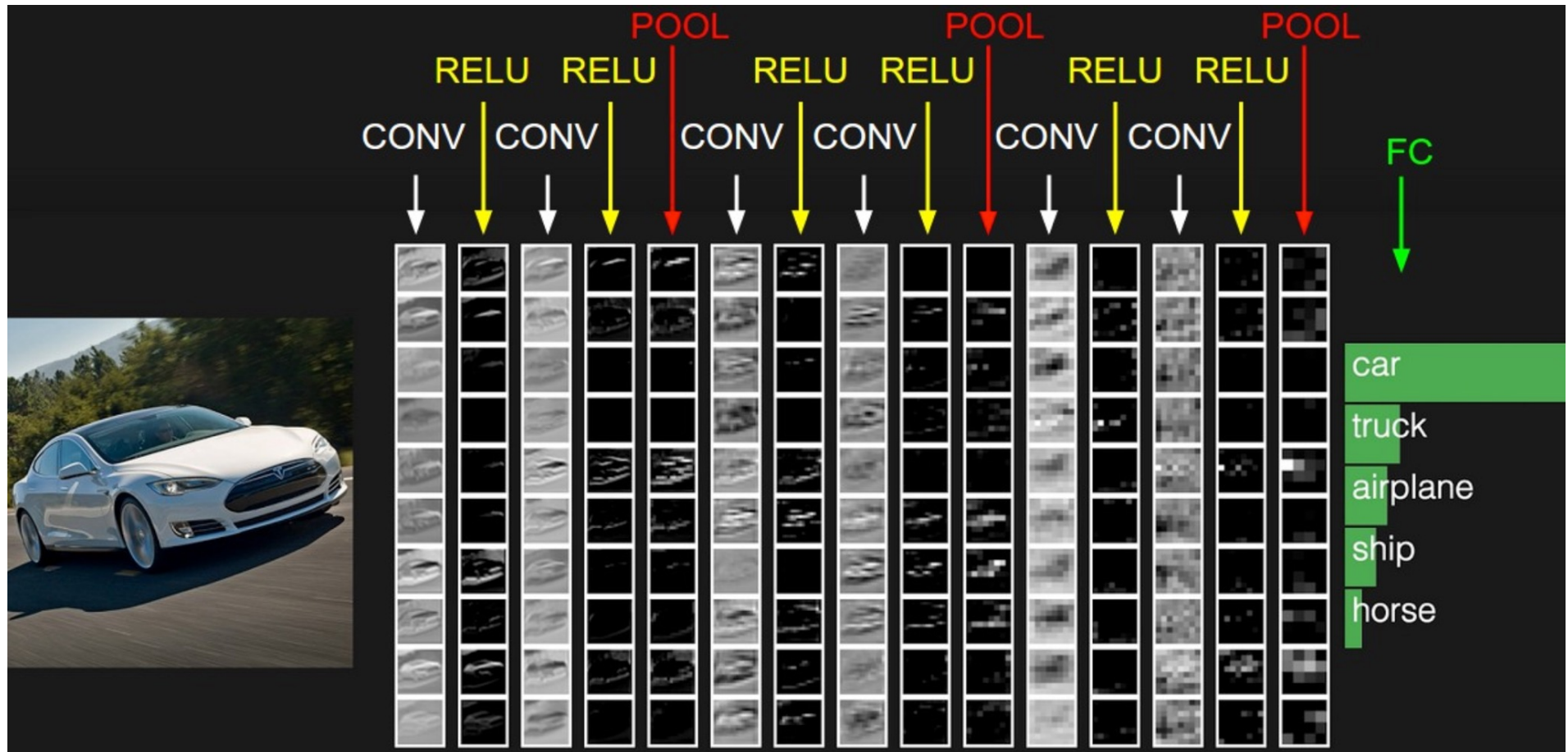


최대풀링(max pooling)



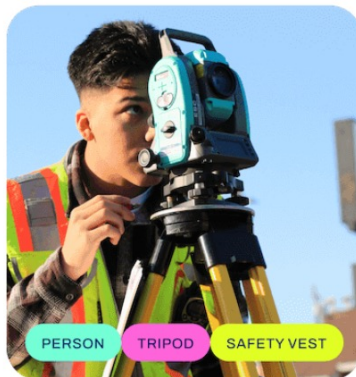
- ▶ 특정한 패턴이 공간상의 어느 위치에 있든 이 활성값이 다음 단계로 넘어가면서 좌우로 조금씩 움직일 수 있다
- ▶ 풀링은 과대적합을 해소하는 데에도 기여한다

이미지 인식

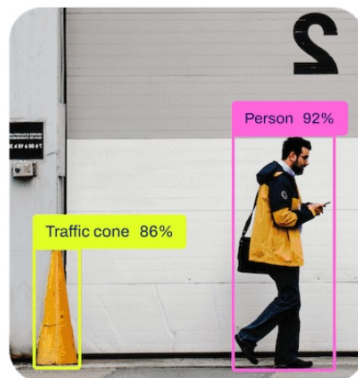


Detect, Segment, Pose

Classify



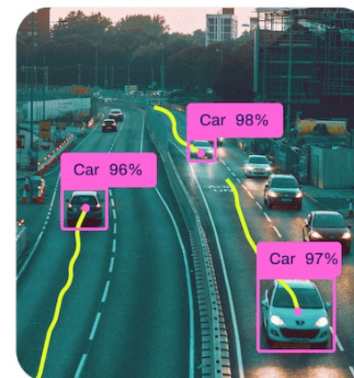
Detect



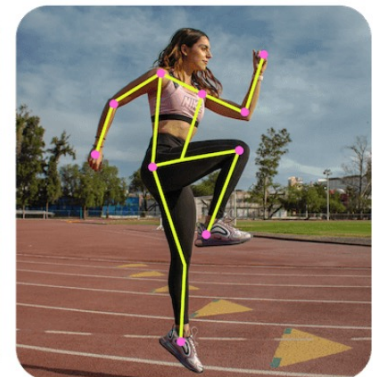
Segment



Track

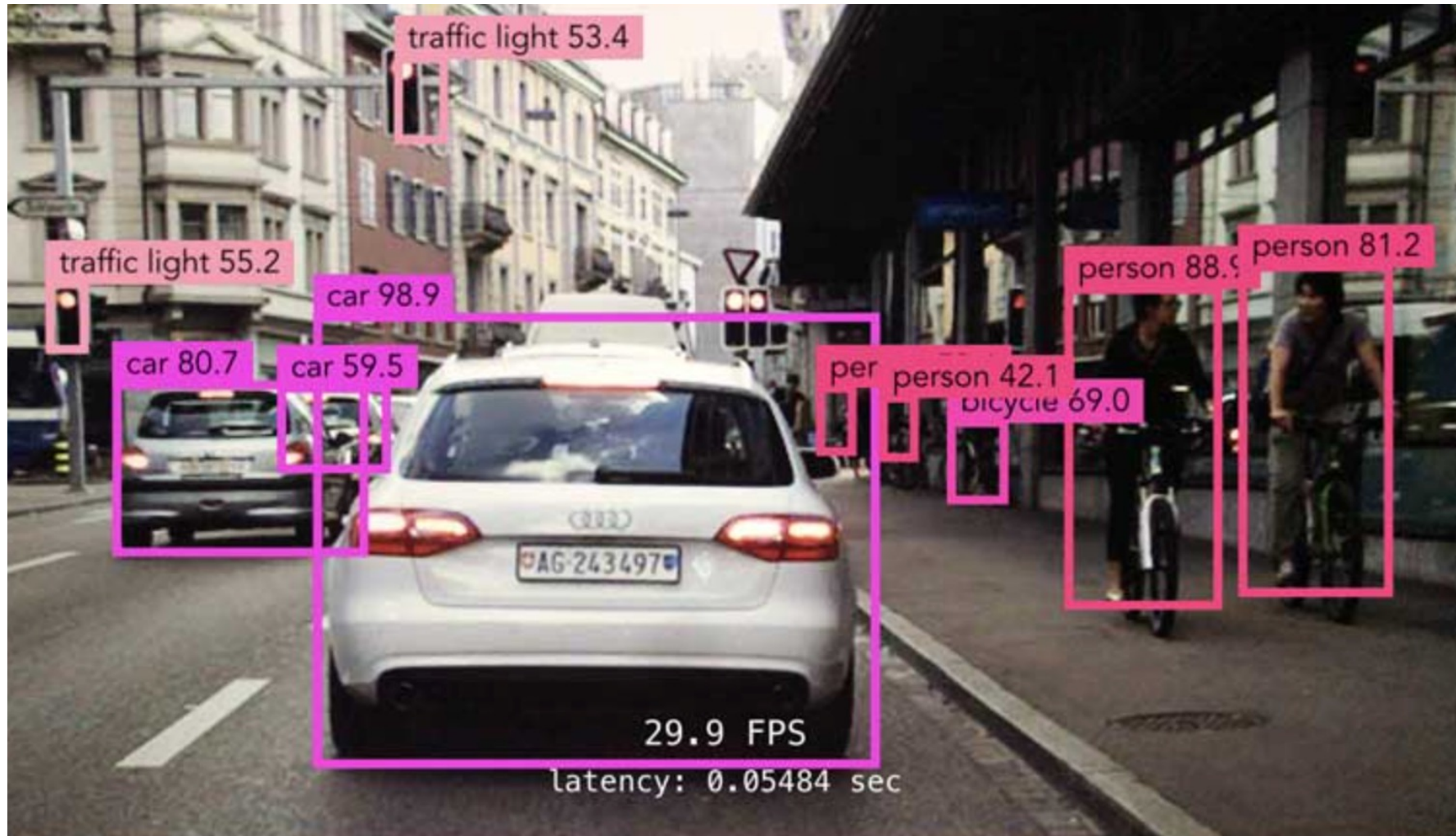


Pose

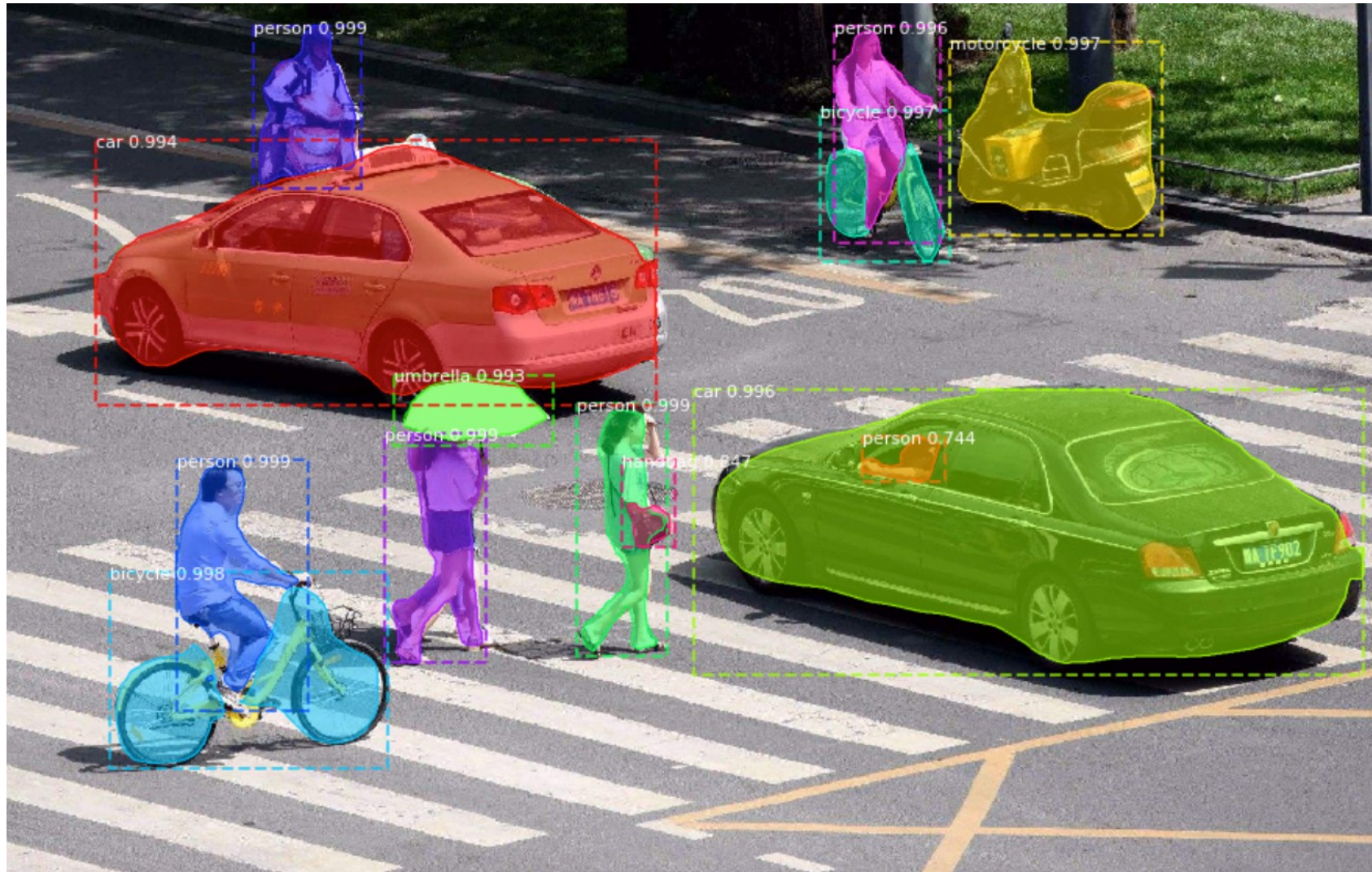


Object Detection

- Confidence score



Object Segmentation

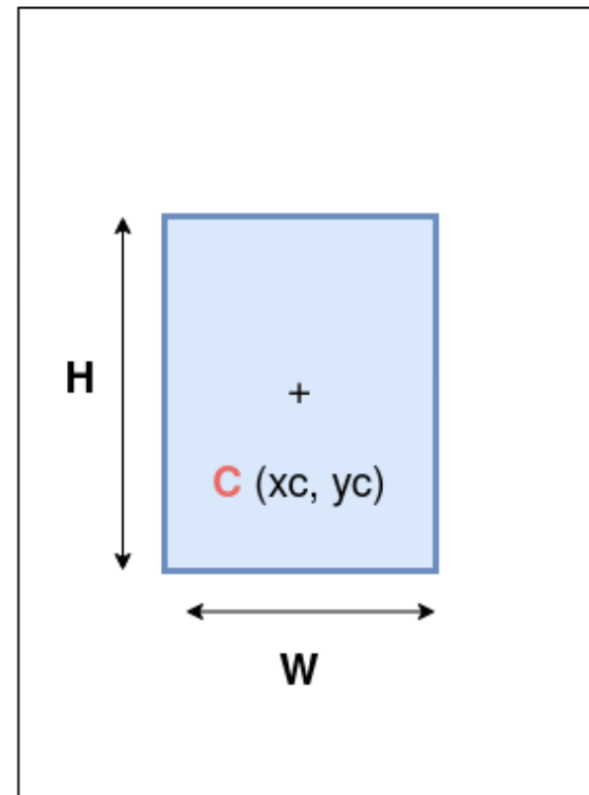
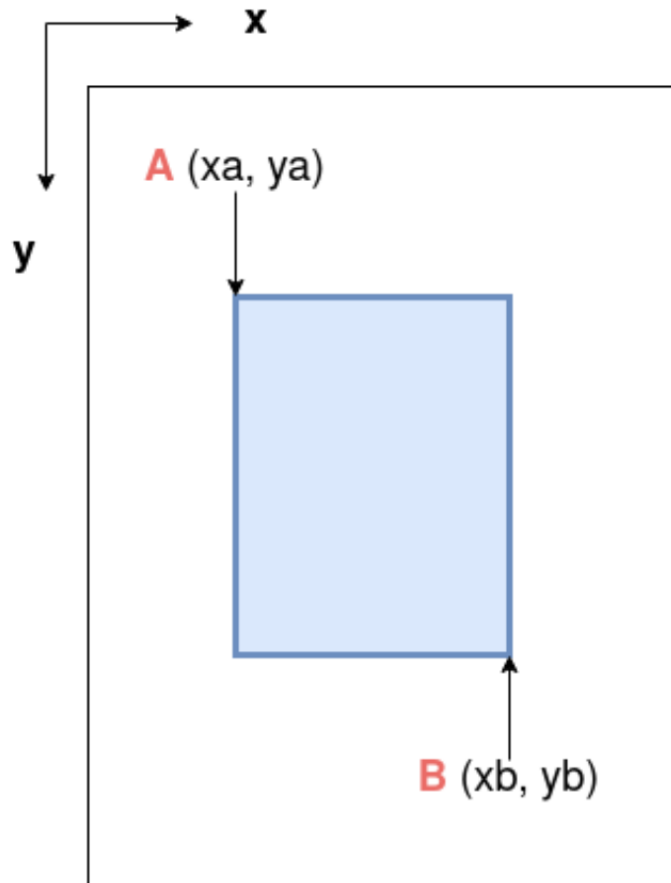


YOLOv8 detect 성능

Model	size (pixels)	mAP ^{val} 50-95	Speed CPU ONNX (ms)	Speed A100 TensorRT (ms)	params (M)	FLOPs (B)
YOLOv8n	640	37.3	80.4	0.99	3.2	8.7
YOLOv8s	640	44.9	128.4	1.20	11.2	28.6
YOLOv8m	640	50.2	234.7	1.83	25.9	78.9
YOLOv8l	640	52.9	375.2	2.39	43.7	165.2
YOLOv8x	640	53.9	479.1	3.53	68.2	257.8

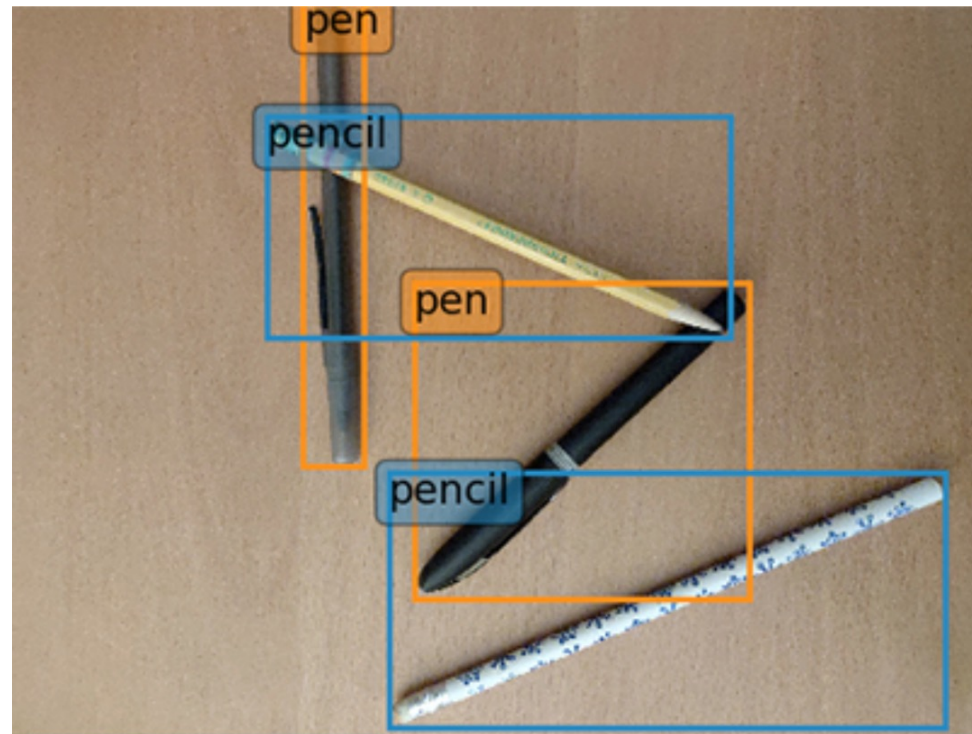
- **mAP^{val}** values are for single-model single-scale on [COCO val2017](#) dataset.
Reproduce by `yolo val detect data=coco.yaml device=0`
- **Speed** averaged over COCO val images using an [Amazon EC2 P4d](#) instance.
Reproduce by `yolo val detect data=coco128.yaml batch=1 device=0|cpu`

xyxy, xywh 포맷

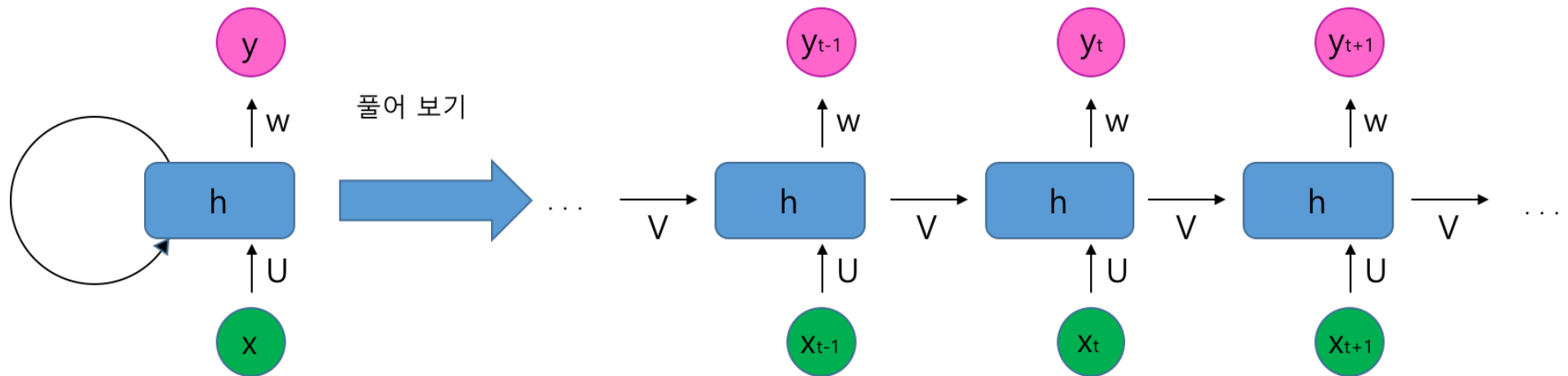


Labeling

- ▶ 모든 객체를 레이블링
- ▶ 짝 차게
- ▶ 클래스는 가능한 세분한 후 나중에 합친다
- ▶ 이미지 크기와 픽셀 고려



RNN의 구조

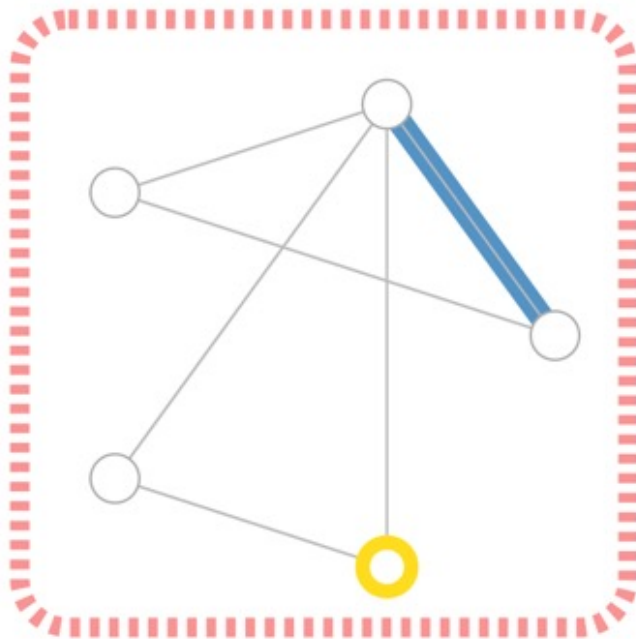


$$h_i = V_{i-1} + UX_i$$
$$y_i = Wh_i$$

- ▶ 위 표현에서 활성화 함수와 bias는 생략됨
- ▶ 상태값 h를 다음 스텝에 사용한다
- ▶ U,V,W는 i의 함수가 아니라 모든 스텝에서 같은 모양을 갖는다

Graph Network

- ▶ Distill
- ▶ 그래프에 포함되는 정보 종류



Vertex (or node) embedding



Edge (or link) attributes and embedding

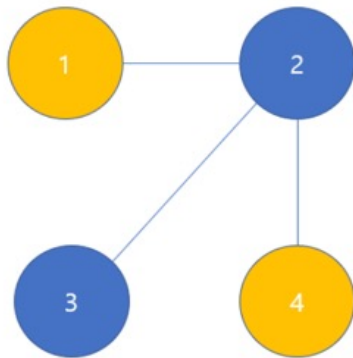


Global (or master node) embedding



Adjacency/Feature Matrix

- ▶ adjacency matrix
 - ▶ node 간의 관계(edge)를 표현하며 아래와 같이 방향성은 없는 경우 대칭적이다
- ▶ feature matrix
 - ▶ node에 담긴 정보를 나타내며, 아래의 경우 특성이 3개라고 가정함 (색을 구분)



$$\text{Adjacency matrix } A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

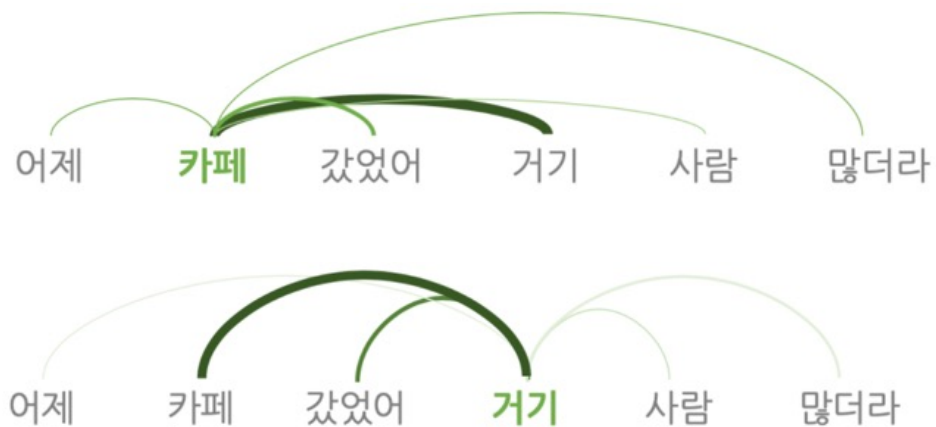
$$\text{Feature matrix } F = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$$

Transformer

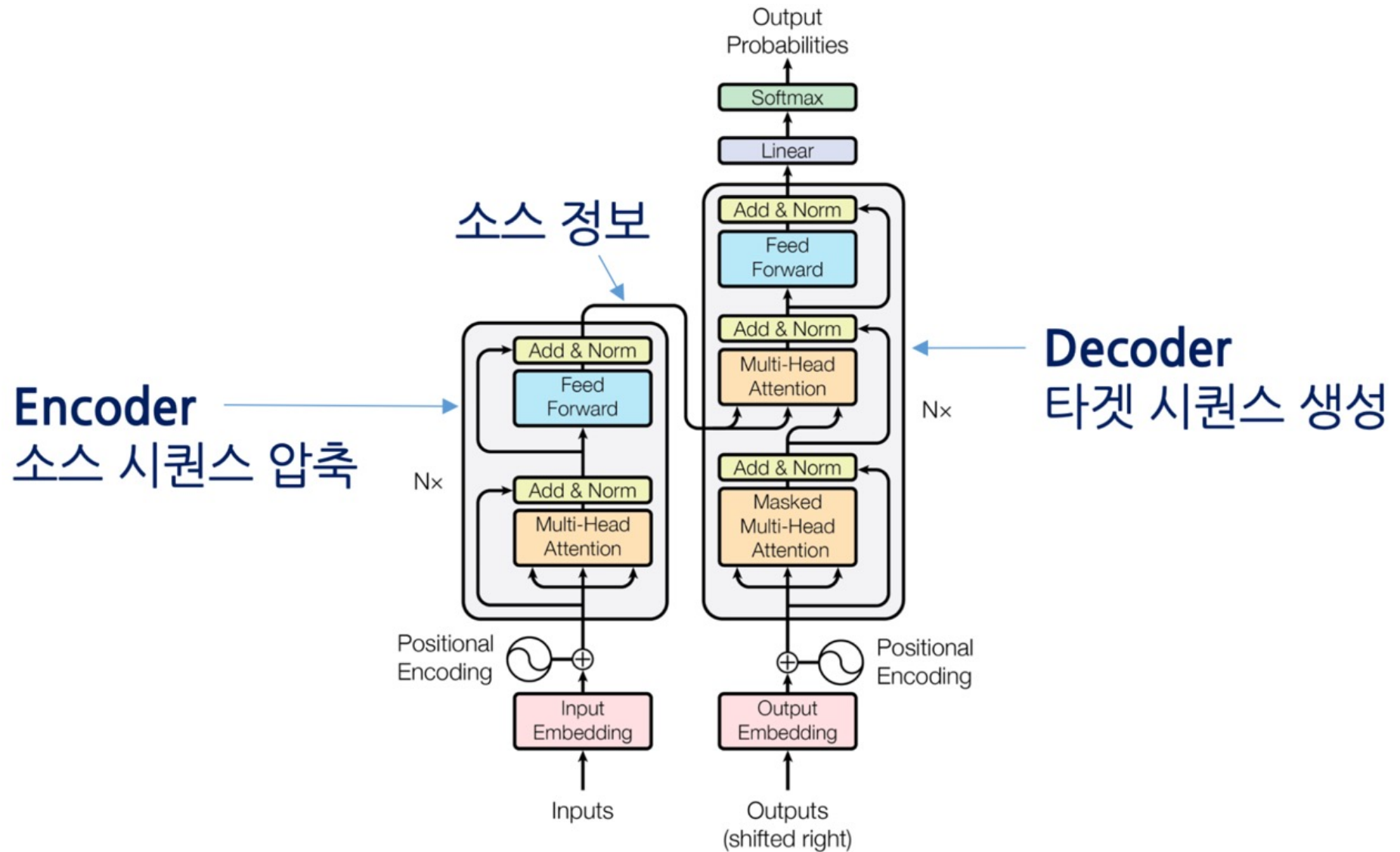
- ▶ 어텐션: café를 디코딩할 때 주의해서 볼 내용은?
 - ▶ RNN 구조에서 동작 seq2seq 모델 사용



- ▶ **셀프 어텐션**: 입력 전체 토큰에 대해 각각 수행하는 어텐션
 - ▶ 트랜스포머는 어텐션(Attention) 기법을 사용한다



트랜스포머



표현 학습

표현 (Representation)

X				y
이름	키	몸무게	나이	운동종목
A	178	73	34	축구
B	169	72	38	탁구
C	173	68	52	농구
...	...	...	...	...
Z	175	82	29	배구

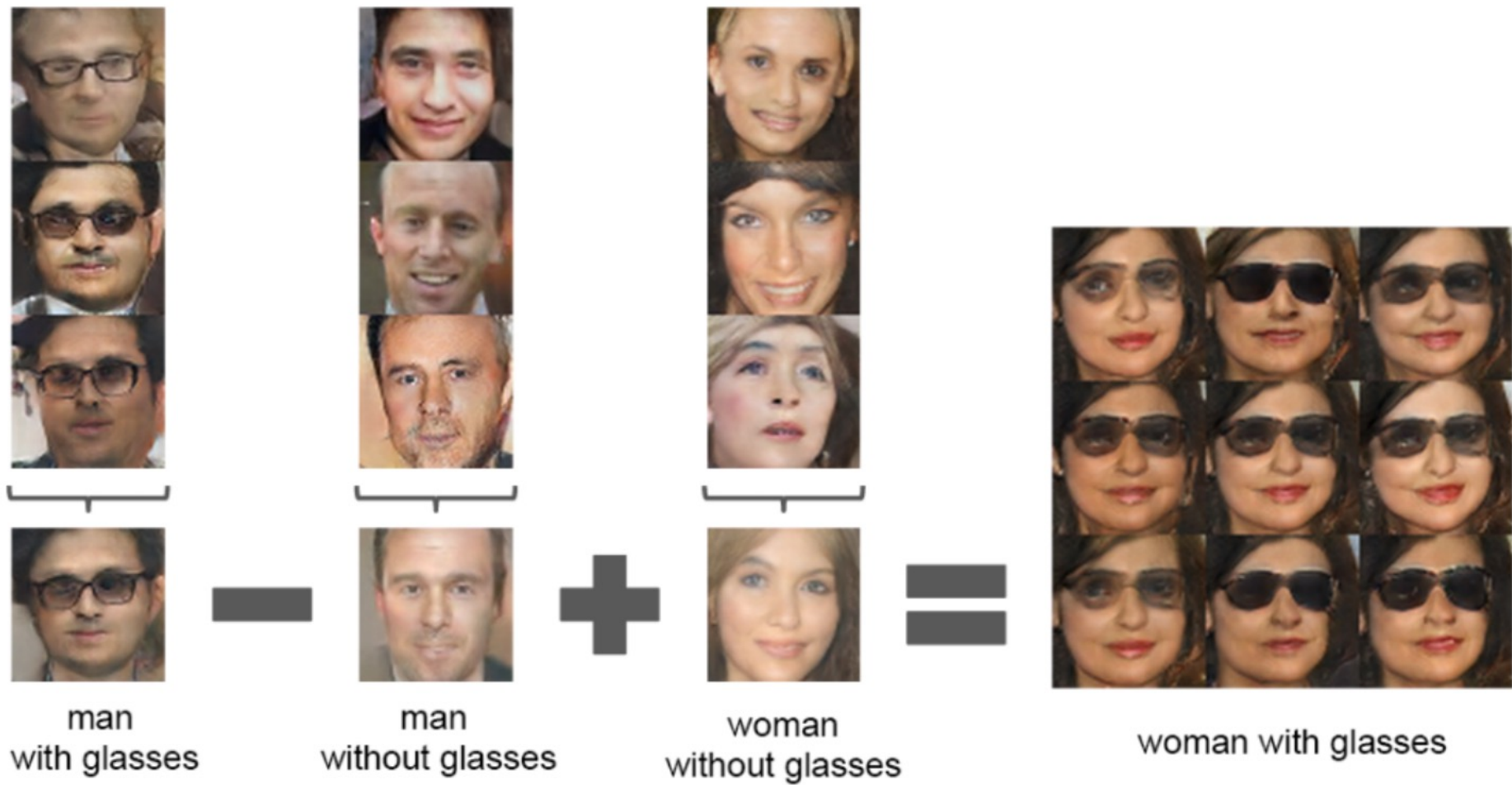
- ▶ X: features, 설명변수, 독립변수, 입력 → **표현(representations)**
- ▶ y: label, 목적변수, 종속변수, 출력

Image Representation



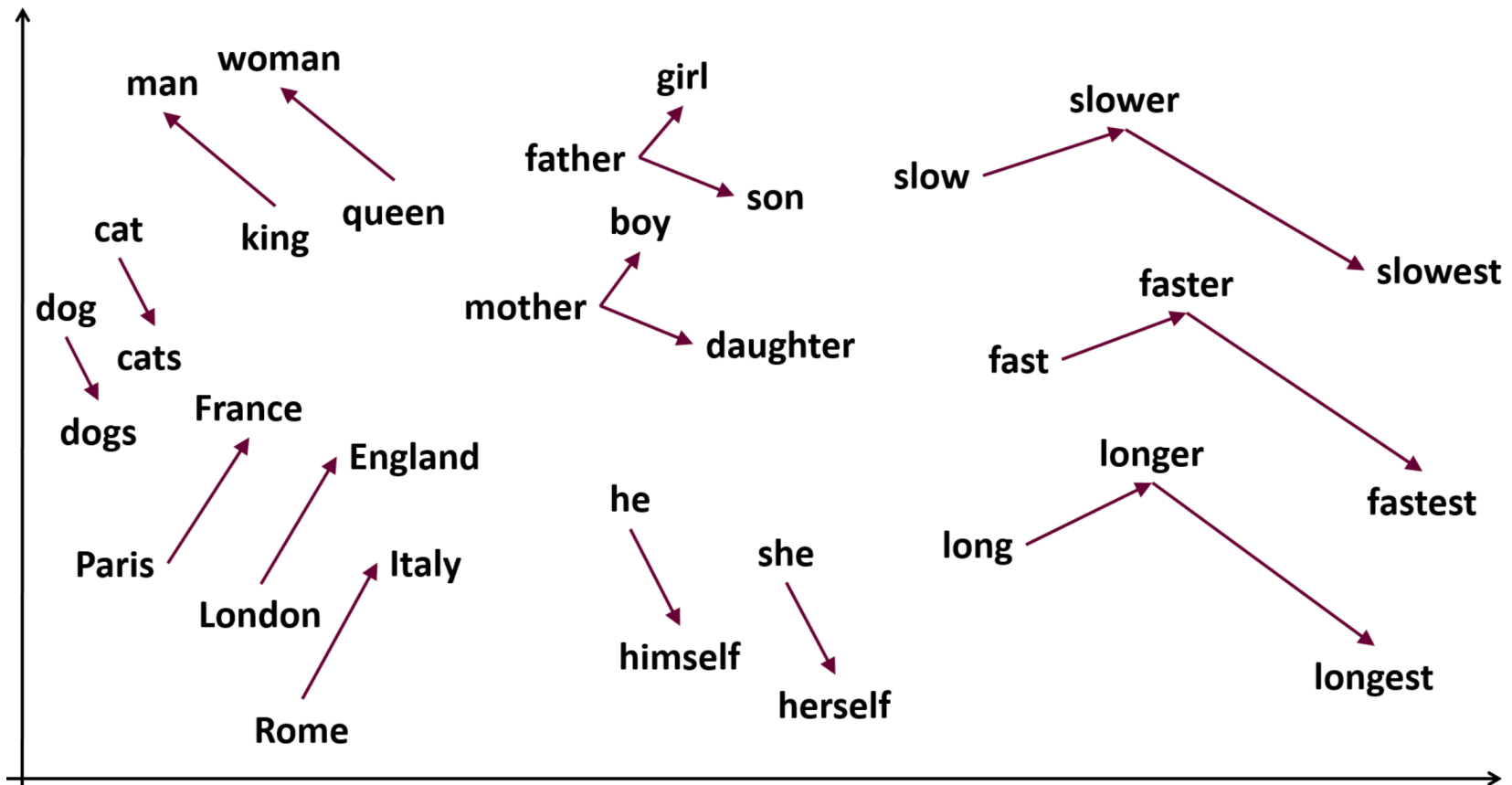
- ▶ 몽타주는 이미지를 수십개의 features(숫자)로 표현하는 것
 - ▶ 안경:0, 눈썹길이:0.8, 눈썹모양:2, 코:0.7, 입술:0.4, 턱:0.4, 귀:3,...
- ▶ 얼굴 = (0, 0.8, 2, 0.7, 0.4, 0.4, 3, ..., 0.6) : vector
 - 이미지를 임베딩 벡터로 표현 가능
 - 벡터 차원을 높이면 매우 정교하게 얼굴을 묘사할 수 있다
 - 벡터 값을 바꾸면 다른 얼굴을 표현할 수 있다 (이미지 생성 모델)

이미지 임베딩 벡터 연산



Word Representation

- ▶ word embedding vector 표현 (300~1000 차원 사용)
- ▶ 단어 사이의 **거리**가 의미가 있다
- ▶ 단어 간의 **방향**이 의미가 있다

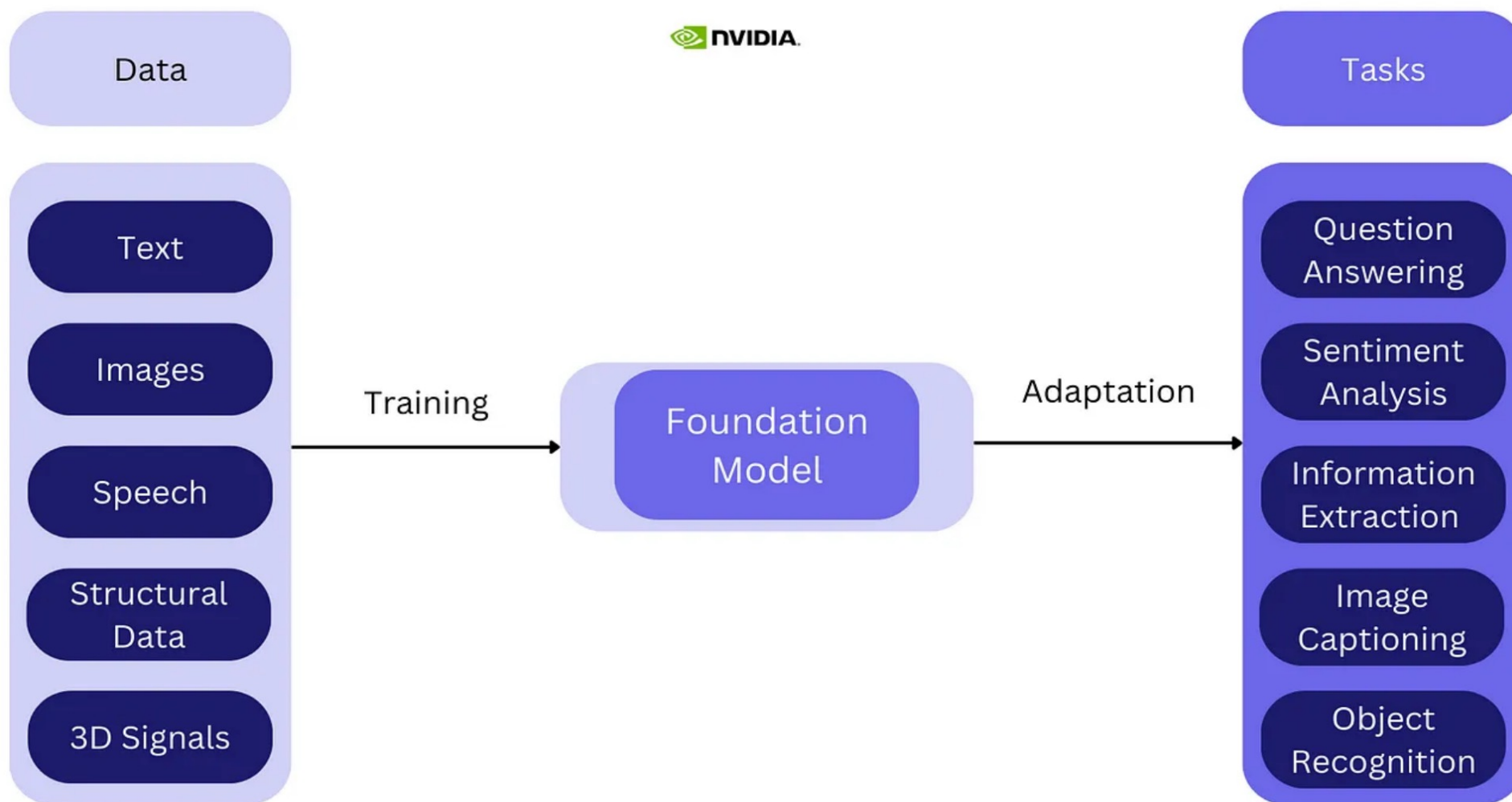


사람-영화 임베딩 벡터

- ▶ 영화 평점 데이터로부터 사람과 영화를 각각 고차원 공간의 벡터로 표현할 수 있다
 - ▶ 비슷한 사람은(영화는) 비슷한 벡터 값(위치)을 갖는다
- ▶ 고차원 벡터를 행렬 분해(Matrix Decomposition)를 하여 저차원으로 줄여 '사람 임베딩'과 '영화 임베딩'을 구할 수 있다
 - ▶ 모든 영화에 대한 평점이 필요한 것은 아니다

				
John	5	1	3	5
Tom	?	?	?	2
Alice	4	?	3	?

Foundation Model



Foundation Model

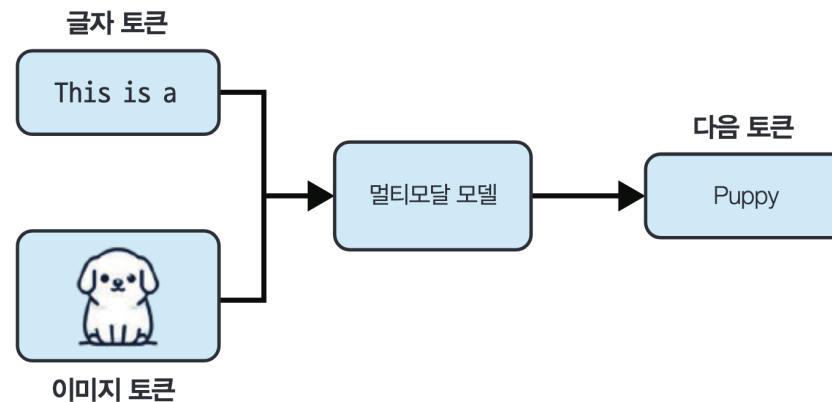
- ▶ 대표적 예: Large Language Model
- ▶ 파운데이션 모델 성능 개선
 - ▶ prompt engineering – 지시문을 효과적으로 제시
 - ▶ RAG(retrieval-augmented generation) – 추가 학습 데이터를 제공
 - ▶ Fine tuning – 정밀 재학습
- ▶ 파운데이션 모델 활용
 - ▶ agentic AI

Language Model

- ▶ 언어 모델(Language Model)
 - ▶ 단어들이 나열된 후 다음에 나타날 가장 적절한 단어들의 확률을 계산하는 모델
 - ▶ 컴퓨터 프로그램도 언어 모델로 작성한다
- ▶ LLM (Large Language Model)
 - ▶ 공개된 대량의 텍스트를 사용하여 사전 학습시킨 모델
 - ▶ Foundation 모델이라고도 한다
 - ▶ BERT(구글), GPT(Generative Pretrained **Transformer**, OpenAI)
- ▶ Bio LLM
 - ▶ 원자 시퀀스로 구성된 분자, 또는 아미노산 시퀀스로 구성된 단백질을 언어 모델로 분석할 수 있다
 - ▶ MoLer (MS) – Structural Motif 생성
 - ▶ BioNeMo(OpenAI) – LLM 기반 DTI
 - ▶ Protein Language Model

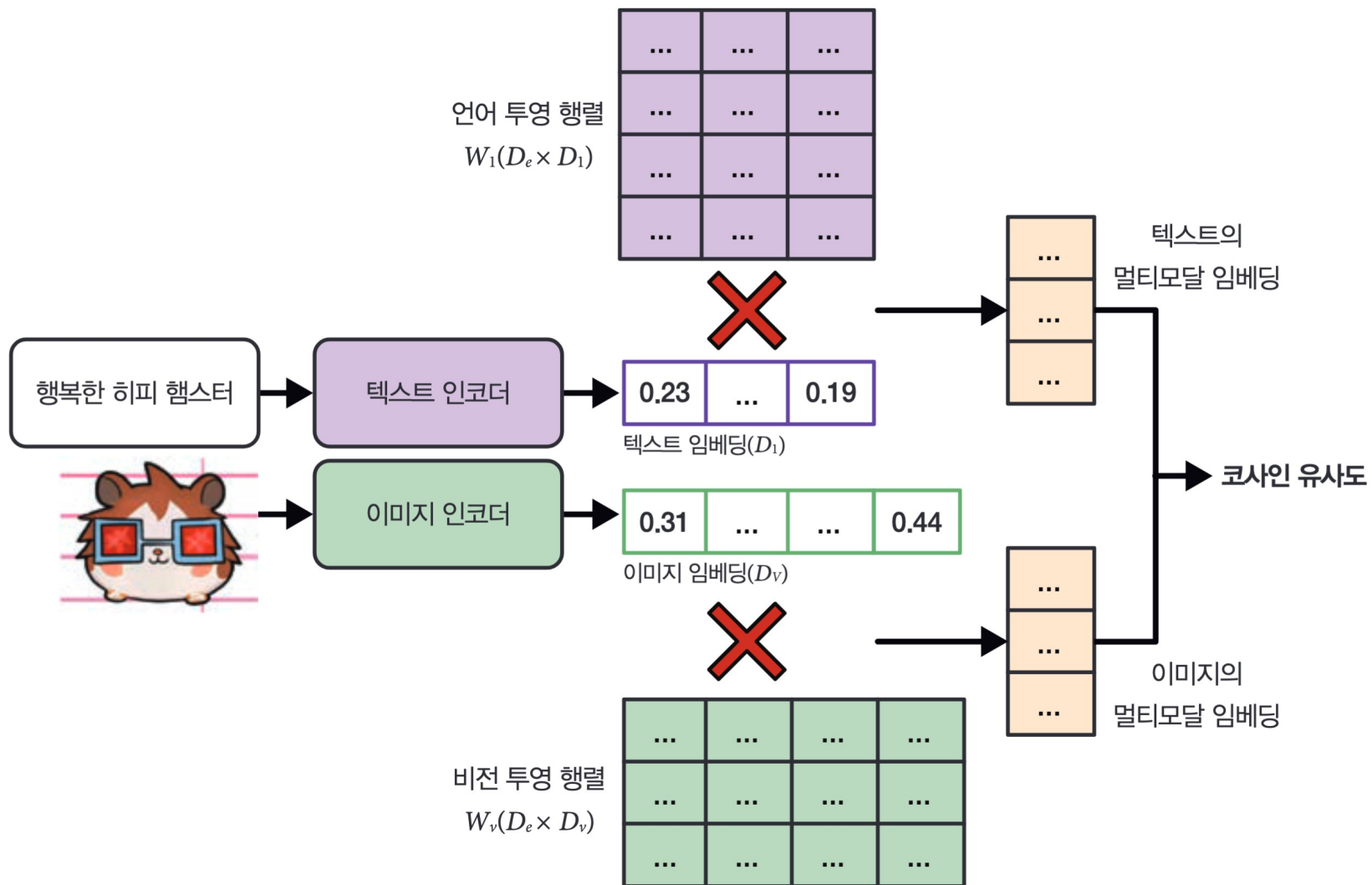
Multi Modality

- ▶ 이미지와 텍스트를 동시에 학습 (2023~)
 - ▶ Large Multimodal Model



- ▶ CLIP (Open AI 2021)
 - ▶ Contrastive Language–Image Pre-training
 - ▶ 추가 학습 없이도 이미지 분류 작업을 수행
 - ▶ 이후 Flamingo, LLaVA, Gemini 등 소개

CLIP



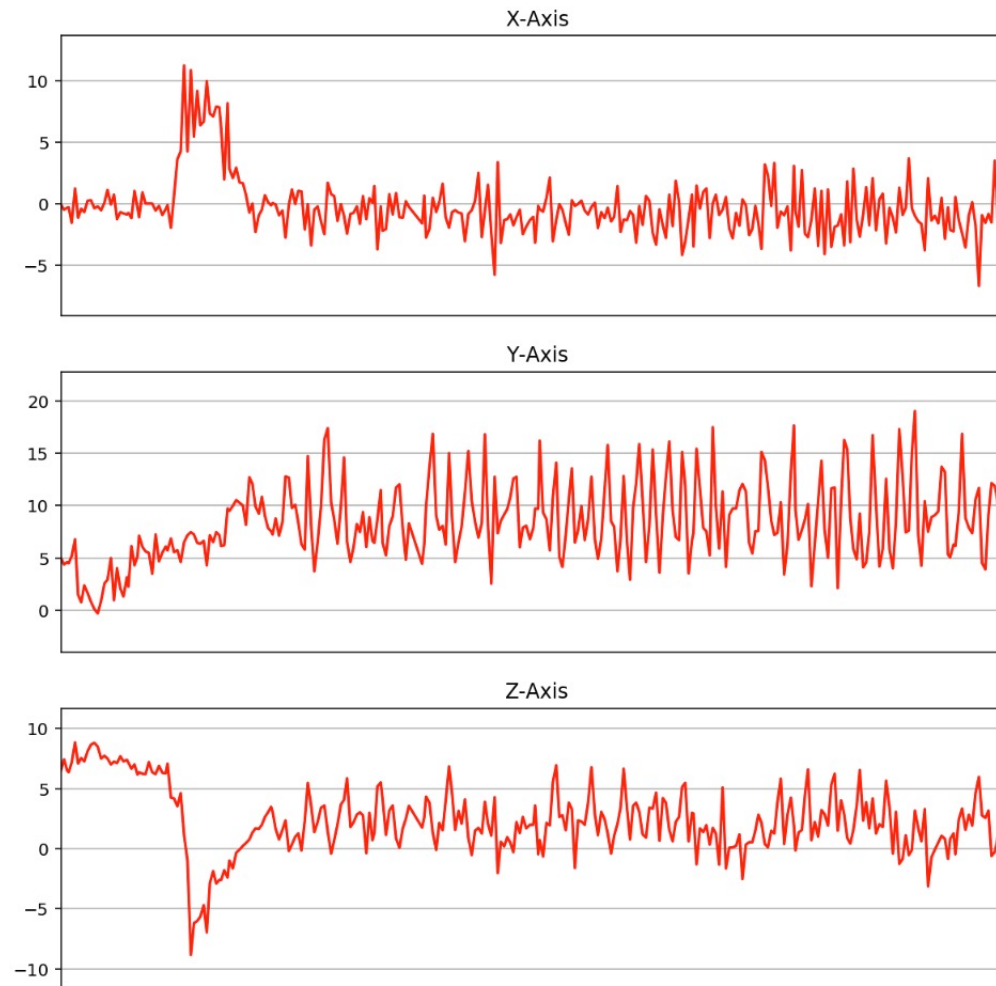
시계열 예측

- ▶ 기본 모델 (naïve model)
- ▶ 선형 모델
- ▶ ARIMA
- ▶ 머신러닝 모델
- ▶ 딥러닝 모델

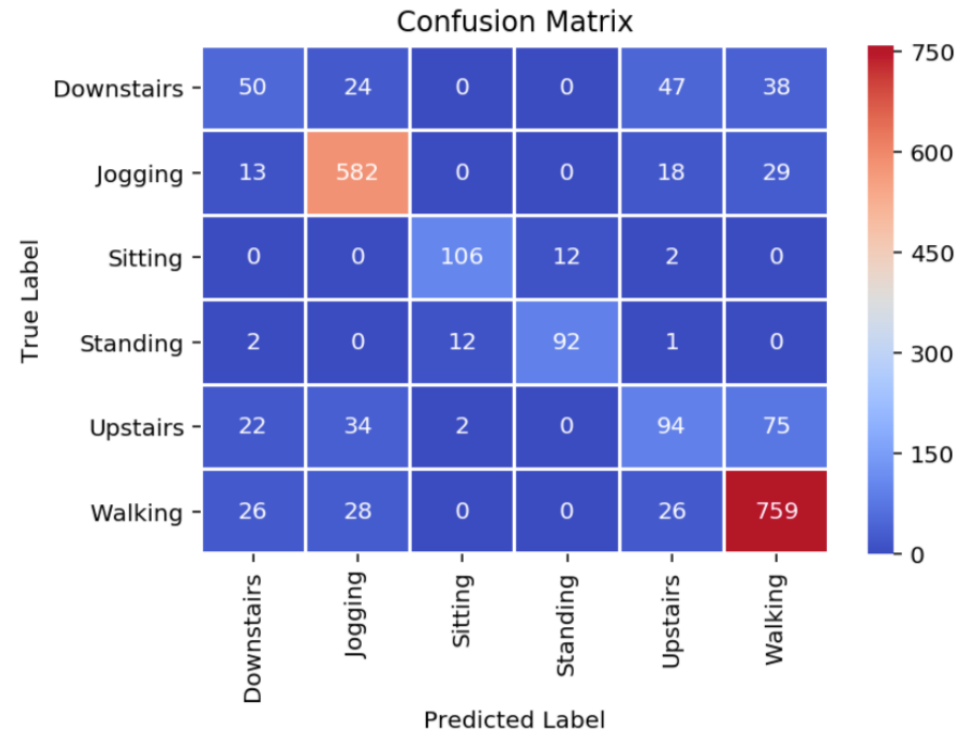
센서 데이터 분석

센서 데이터 패턴 분류

계단내려가기



분류 결과

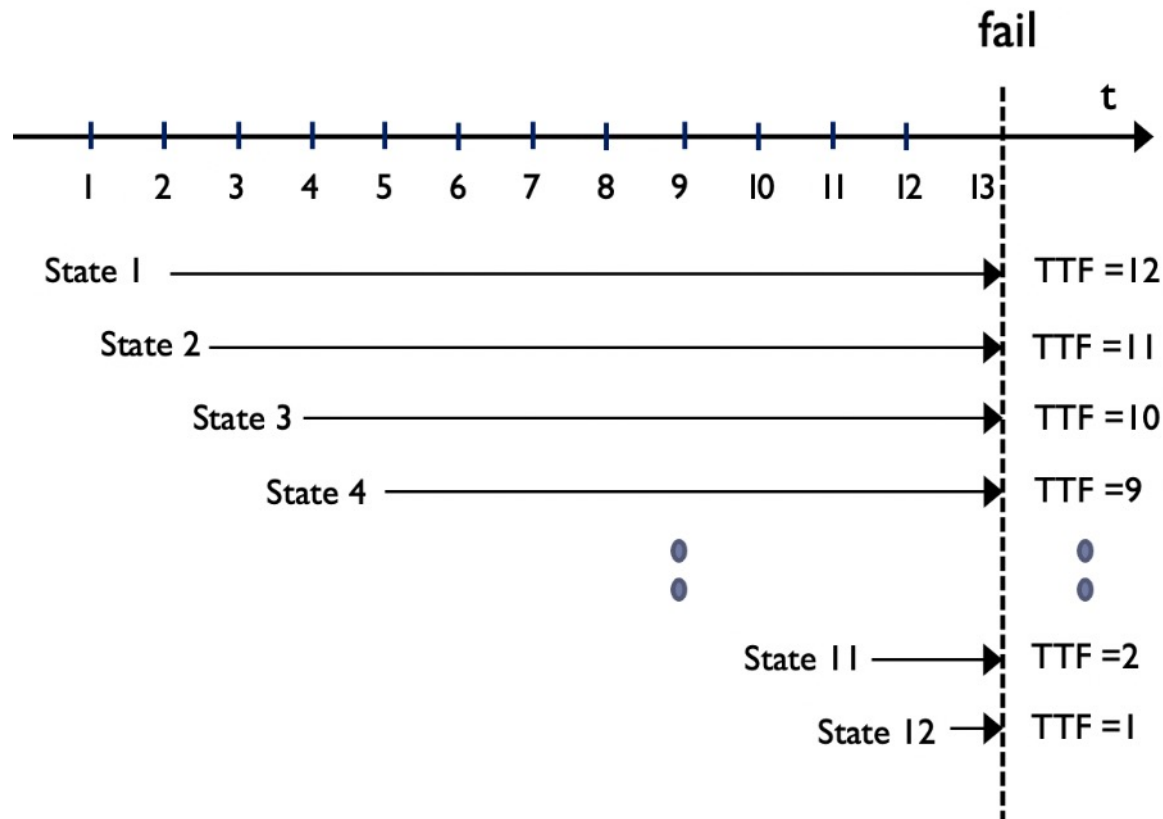


	precision	recall	f1-score	support
0	0.44	0.31	0.37	159
1	0.87	0.91	0.89	642
2	0.88	0.88	0.88	120
3	0.88	0.86	0.87	107
4	0.50	0.41	0.45	227
5	0.84	0.90	0.87	839
avg / total	0.79	0.80	0.79	2094

예지 보수

PM 회귀 모델

- ▶ TTF(time to fail): 고장까지 남은 시간을 예측

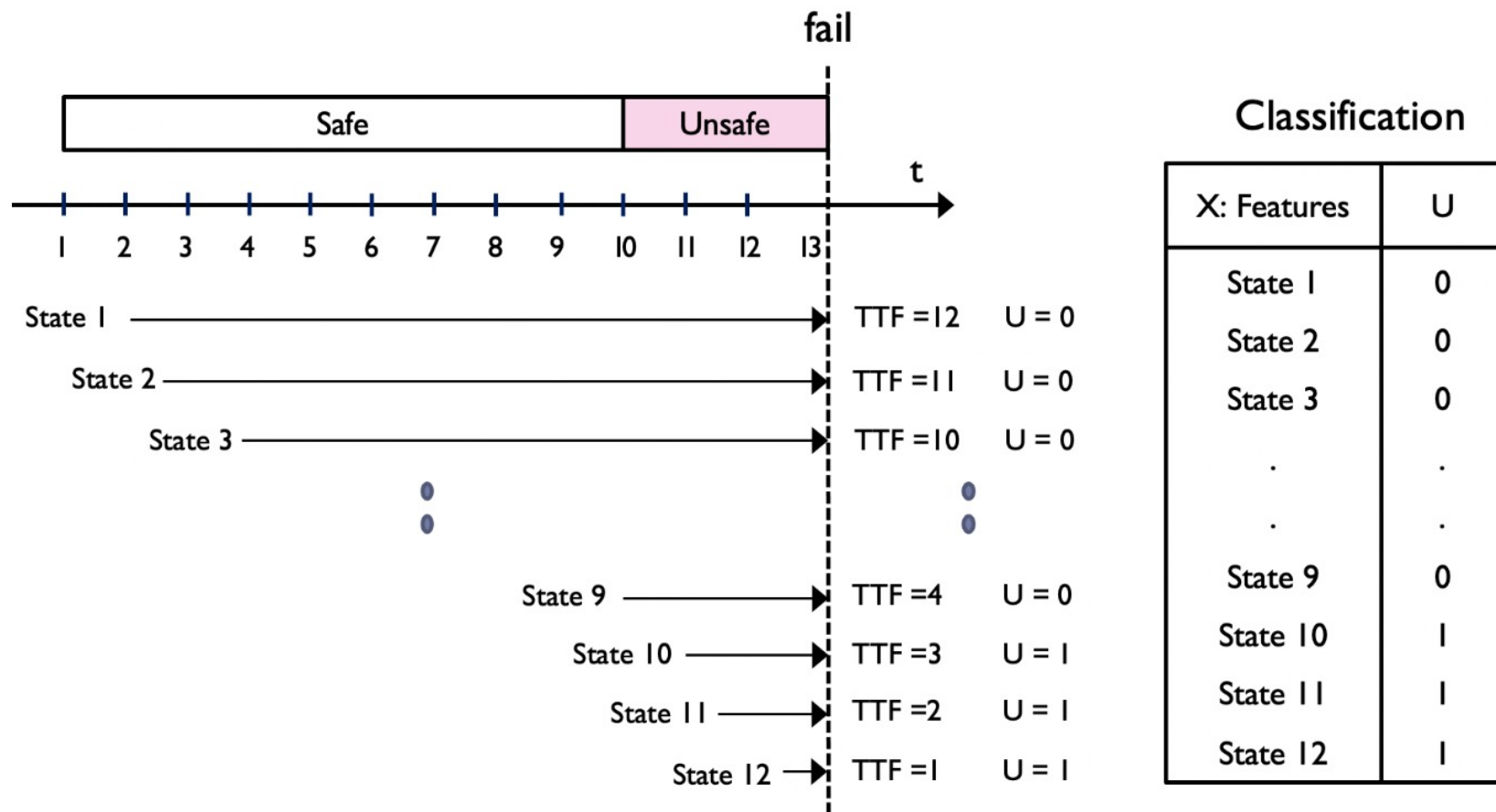


Regression

X: Features	y: <u>TTF</u>
State 1	12
State 2	11
State 3	10
State 4	9
.	.
.	.
State 11	2
State 12	1

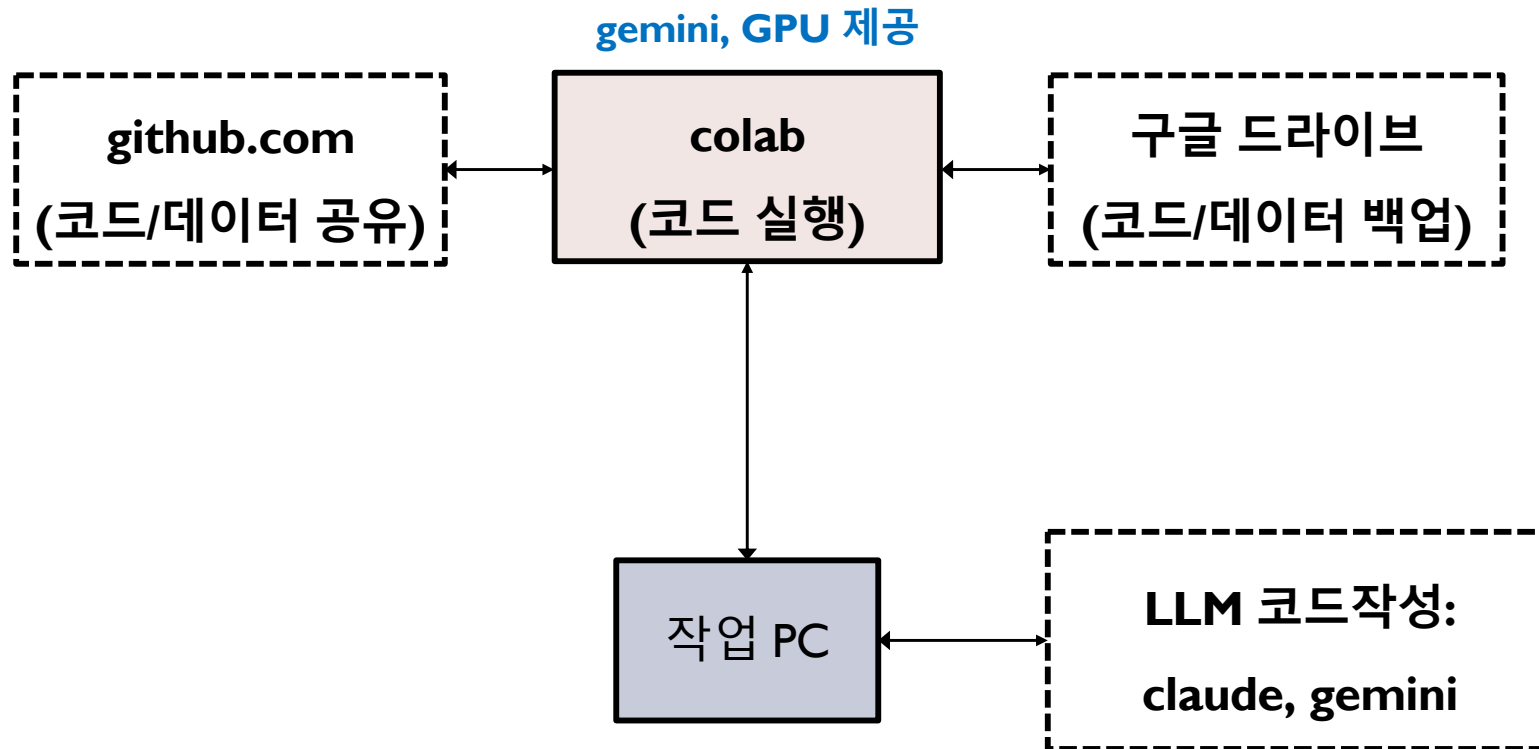
PM 분류 모델

- ▶ TTF가 주어진 경계치 이하로 낮은 위험한 상태를 예측



실습

실습 환경



참고 예제

파이썬 (1~5)

데이터 탐색과 전처리 (6~8)

머신러닝 모델 (9~15)

특성공학 (16~17)

클러스터링 (18)

딥러닝 모델 (19)

이미지 분석 (20~21)

시계열 분석 (22~26)

예지보수 (27)

객체 검출 (28~33)

텍스트 분석 (34~36)

예제 1

- ▶ j_01_Python.ipynb
- ▶ j_02_Pandas.ipynb
- ▶ j_03_Numpy.ipynb
- ▶ j_04_Function.ipynb
- ▶ j_05_FileFolder.ipynb
- ▶ j_06_Graphs.ipynb
- ▶ j_07_Preprocessing.ipynb
- ▶ j_08_Preprocessing_Example.ipynb
- ▶ j_09_MachineLearning.ipynb
- ▶ j_10_Regression.ipynb
- ▶ j_11_Classification.ipynb
- ▶ j_12_LogisticRegression.ipynb

예제 2

- ▶ j_13_DT_RF_Boost_Titanic.ipynb
- ▶ j_14_DT_RF_Boost_Life.ipynb
- ▶ j_15_SVM.ipynb
- ▶ j_16_PCA.ipynb
- ▶ j_17_Add_features.ipynb
- ▶ j_18_Clustering.ipynb
- ▶ j_19_DeepLearning.ipynb
- ▶ j_20_CatDog.ipynb
- ▶ j_21_TransferLearning.ipynb
- ▶ j_22_ActivityRecognition.ipynb
- ▶ j_23_TimeseriesForecasting.ipynb
- ▶ j_24_TimeSeries_XY_bike.ipynb

예제 3

- ▶ j_25_Prophet.ipynb
- ▶ j_26_LSTM.ipynb
- ▶ j_27_PM.ipynb
- ▶ j_28_YOLOv8_run.ipynb
- ▶ j_29_YOLOv8_images.ipynb
- ▶ j_30_YOLOv8_images_PC.ipynb
- ▶ j_31_YOLOv8_Aquarium_Data.ipynb
- ▶ j_32_YOLOv8_Aquarium_Data_PC.ipynb
- ▶ j_33_OCR_PC.ipynb
- ▶ j_34_Text_BoW.ipynb
- ▶ j_35_SentimentAnalysis.ipynb
- ▶ j_36_TopicAnalysis.ipynb
- ▶ j_37_Streamlit.py
- ▶ j_38_AutoEncoder.ipynb

Q & A